CAMADA FÍSICA DA COMPUTAÇÃO · INSPER · 2026.2

Projeto 6 — DTMF

Teclas de telefone com senoides e a Transformada de Fourier

Como gerar o som de uma tecla de telefone somando duas senoides, enviar esse som pelo ar de um computador a outro e descobrir, do outro lado, qual tecla foi apertada olhando o sinal no domínio da frequência.

> **Guia de estudo para o quiz.** Este relatório explica o projeto do início ao fim: o que o enunciado pede, os conceitos por trás de cada linha de código (som, digitalização, Nyquist, Fourier, FFT, janelas, picos, DTMF), como a solução foi feita e testada, a lista de Digitalização resolvida e uma bateria de perguntas de revisão com respostas.

Relatório explicativo · feito a partir do enunciado, dos slides de Fourier (aula 16), da lista de Digitalização e do código-fonte · outubro de 2026

Sumário

1 O repositório e a disciplina . . . 3

2 O que o enunciado pede . . . 4

3 Conceitos fundamentais . . . 6

4 A solução: visão geral . . . 14

5 Emissor passo a passo . . . 15

6 Receptor passo a passo . . . 18

7 Resultados e testes . . . 20

8 Como interpretar os gráficos . . . 22

9 Como rodar e roteiro de apresentação . . . 23

10 Lista de exercícios — Digitalização . . . 25

11 Revisão para o quiz . . . 27

A Apêndice: código completo . . . 30

RESUMO-RELÂMPAGO

O projeto em seis frases

1. Cada tecla do telefone é a **soma de duas senoides**: uma frequência da linha (697, 770, 852 ou 941 Hz) e uma da coluna (1209, 1336, 1477 ou 1633 Hz).
2. O **emissor** gera as duas senoides com 44 100 amostras por segundo, soma, divide por 2 para não saturar e toca pela placa de som.
3. O som atravessa o ar e é **digitalizado** pelo microfone do receptor (amostragem + quantização).
4. No tempo, a soma parece uma onda bagunçada; a **Transformada de Fourier (FFT)** mostra o mesmo sinal como dois picos, exatamente nas frequências que o compõem.
5. O **receptor** acha os picos (pelo menos 5, ajustando a sensibilidade), escolhe o mais forte perto de cada grupo DTMF e consulta a tabela para descobrir a tecla.
6. Conceitos que caem: Nyquist ($f_s > 2 \cdot f_{max}$), resolução $\Delta f = 1/T$, janela de Hamming contra vazamento, clipping cria frequências falsas, só metade do espectro importa.

1 O repositório e a disciplina

Onde o Projeto 6 se encaixa na sequência de projetos

O repositório **Camada-Fisica-da-Comp-** tem uma pasta por projeto da disciplina Camada Física da Computação (Insper, prof. Rodrigo Carareto). Os projetos 1 a 4 montam, de baixo para cima, uma pilha de comunicação serial em Python, com o Arduino servindo só como conversor USB–serial. O 5 desce um nível e gera os bits na mão dentro do Arduino. O 6 muda o meio: **não há mais fio**; o canal é o ar e o sinal é som.

#	Pasta	O que o projeto faz	Conceito central
1	Projeto1_LoopBackUART	Envia uma imagem pela UART e recebe de volta no mesmo PC (loop-back).	Camadas aplicação → enlace → interface física; bytes crus.
2	Projeto2_PontoFlutuante	Cliente envia números IEEE-754 de 32 bits; servidor devolve a soma; time-out de 5 s.	Codificar dados em bytes; cliente/servidor em 2 PCs.
3	Projeto3_DataGrama	Transfere arquivos em datagramas (head + payload + EOP), com handshake, confirmação e retomada.	Protocolo, fragmentação, overhead.
4	Projeto4_CRC	O mesmo protocolo com CRC-16 do payload, pedido de reenvio e logs.	Detecção de erros.
5	Projeto5_Serializacao	Dois Arduinos trocam caracteres gerando o frame UART bit a bit em um pino comum.	Serialização: start, dados, paridade, stop, baud rate.
6	Projeto6_DTMF	Gera o som de uma tecla de telefone e reconhece a tecla pelo microfone.	Sinal analógico, digitalização, Fourier.

Pelo site da disciplina, depois deste vêm o **Projeto 7 (Filtro Digital)** e o **Projeto 8 (Modulação AM)**, e os dois dependem de Fourier: um filtro é projetado dizendo quais frequências passam, e a modulação AM desloca o espectro de um sinal. Ou seja, o que se aprende aqui é a base dos próximos. Cada projeto é apresentado em sala, o quiz do projeto só é liberado depois da apresentação, e a média final é 50% prova + 50% média dos quizzes.

IDEIA-CHAVE

Até o Projeto 5 a informação viajava como **níveis de tensão** num fio (0 V / 5 V): um sinal digital, lido bit a bit. No Projeto 6 ela viaja como **vibração do ar**: um sinal analógico. E a tecla não está em nenhum valor instantâneo do sinal: ela está nas **frequências** que o compõem. Por isso o receptor precisa da Transformada de Fourier.

Os arquivos da pasta Projeto6_DTMF

Arquivo	Origem	Papel
Enunciado projeto 06 DTMF.pdf	professor	O que o projeto pede (seção 2).
Digitalização.pdf	professor	Texto + lista de exercícios a entregar resolvida (seção 10).
fourier.pptx	professor	Slides da aula 16: transformada contínua, DFT e exemplos em Python.
encode_versaoAlunos.py decode_versaoAlunos.py	professor	Modelos com o roteiro em comentários. Mantidos intactos, para comparação.
suaBibSignal.py	novo	Biblioteca de apoio: tabela DTMF, gerar senoide, calcular a FFT.

Arquivo	Origem	Papel
encode.py	novo	Emissor: pergunta a tecla, gera, toca e plota o tom.
decode.py	novo	Receptor: grava, faz a FFT, acha os picos e identifica a tecla.
Relatorio_Projeto6_DTMF.pdf	novo	Este documento.

PEÇA QUE FALTAVA

Os modelos importam `suaBibSignal`, mas essa "biblioteca de apoio (cedida)" **não estava no repositório**. Ela foi reescrita com a mesma interface (`signalMeu`, `generateSin`, `calcFFT`, `plotFFT`) e três correções em relação à versão que circula nas turmas:

- `scipy.signal.hamming` foi **removida no SciPy 1.13**; o código antigo quebra com "has no attribute hamming". Agora usa `np.hamming`.
- O eixo de frequências usa `np.fft.rfftfreq`, que dá exatamente $k \cdot f_s/N$ (o `linspace(0, fs/2, N//2)` antigo fica um pouco desalinhado).
- O tempo usa `np.arange(n)/fs`: passo exato de $1/f_s$.

2 O que o enunciado pede

E onde cada requisito foi atendido no código

O enunciado pede para implementar em software um sistema **DTMF** (*dual-tone multi-frequency*), o sinal que as companhias telefônicas usam para saber qual tecla foi digitada. Uma aplicação toca o som da tecla; outra, em outro computador ou celular, capta esse som pelo microfone e descobre a tecla pela Transformada de Fourier.

	1209 Hz	1336 Hz	1477 Hz	1633 Hz
697 Hz	1	2	3	A
770 Hz	4	5	6	B
852 Hz	**7**	8	9	C
941 Hz	*	0	#	D

Teclado DTMF (padrão ITU-T Q.23). Linha = frequência baixa, coluna = frequência alta. Em destaque, a tecla 7 = 852 Hz + 1209 Hz, usada nos exemplos.

ERRO DE DIGITAÇÃO NA TABELA DO ENUNCIADO

A tabela do enunciado traz **679 Hz** e **825 Hz**. O padrão DTMF (ITU-T Q.23) é **697 Hz** e **852 Hz**: são os mesmos números com dois dígitos trocados. A prova está no próprio enunciado: o exemplo de saída imprime "*Frequências : 1209 + 852*" para a tecla 7. A tecla **X** da tabela é o ***** (asterisco). O código usa o padrão.

No quiz, raciocine pela estrutura (linha = frequência baixa, coluna = alta) em vez de decorar números: assim você acerta mesmo que a questão use a tabela do enunciado.

Lado emissor

Requisito	Como foi atendido	Onde
Perguntar ao usuário qual número (0 a 9) quer digitar	`input()` em laço; aceita também A–D, * e # (e X como sinônimo de *). Digitar q sai.	pedir_tecla
Emitir por alguns segundos as duas frequências	Duas senoides de amplitude 1, f_s = 44 100 Hz, 5 s, somadas e tocadas com `sd.play`.	gerar_tom, emitir
Plotar no tempo as duas frequências somadas	Três gráficos: a senoide baixa, a alta e a soma (20 ms).	plotar
Plotar o Fourier do sinal emitido	`calcFFT` com janela de Hamming, 0 a 2000 Hz, picos rotulados.	plotar
Opcional: salvar o sinal em arquivo	`--salvar` grava `tons/tecla_7.wav` (16 bits).	salvar_wav

Lado receptor

Requisito	Como foi atendido	Onde
Captar o áudio pelo microfone, sem gravar silêncio	Contagem regressiva, depois `sd.rec`; o silêncio das pontas é cortado automaticamente.	gravar, trecho_com_som
Fazer o Fourier do sinal captado	`calcFFT` (Hamming + FFT).	analisar
Identificar os picos: no mínimo 5, ajustando a sensibilidade	`peakutils.indexes` com `thres` diminuindo até aparecerem 5 picos.	detectar_picos
Identificar a tecla e "printar". Cuidado!	Pico mais forte de cada grupo, a até 2% da frequência DTMF e acima do piso de ruído; consulta a tabela.	identificar_tecla
Plotar no tempo o sinal recebido	Gravação inteira (com o trecho usado destacado) + zoom de 20 ms.	plotar
Plotar o Fourier do sinal recebido	Com os picos marcados e as 8 frequências DTMF.	plotar

Critérios de avaliação

O enunciado lista três níveis, do mais básico ao completo:

1. **Produzir o sinal corretamente**, mesmo sem reconhecer a tecla.
2. **Reconhecer no receptor a tecla digitada em todas as tentativas.**
3. **Reconhecer a tecla, plotar os gráficos corretamente e saber interpretá-los** (a seção 8 prepara para isso).

Além disso, a entrega é a apresentação presencial do projeto funcionando **e a lista de exercícios de Digitalização resolvida** (seção 10).

3 Conceitos fundamentais

Da onda sonora ao número no computador, e de volta

3.1 Som, transdutores e o caminho do sinal

Som é uma perturbação mecânica: um corpo vibra, empurra o ar, e a variação de pressão se propaga porque o ar é elástico. Um **transdutor** converte uma grandeza física em outra. No projeto há dois:

- **Alto-falante** (elétrico → mecânico): a corrente passa por uma bobina presa ao cone, dentro de um ímã; a força magnética faz o cone vibrar e empurrar o ar.
- **Microfone** (mecânico → elétrico): o som faz vibrar um diafragma com uma bobina num campo magnético; o fluxo variando induz uma força eletromotriz (lei de Faraday) que copia a variação de pressão.

Entre o Python e esses transdutores está a **placa de som**: um **DAC** (conversor digital → analógico) na saída e um **ADC** (analógico → digital) na entrada. O caminho completo é:

O CAMINHO DO SINAL

números no Python → DAC → tensão → alto-falante → **pressão no ar** → microfone → tensão → ADC → **números no Python**. O emissor trabalha na primeira metade, o receptor na segunda, e no meio o sinal é analógico.

3.2 A senoide

$$s(t) = A \sin(2\pi f t + \varphi)$$

Símbolo	Nome	Significado	No projeto
A	amplitude	Altura da onda; no som, o volume.	1 (o enunciado pede)
f	frequência	Ciclos por segundo, em hertz (Hz); no som, o tom (grave/agudo).	852 Hz e 1209 Hz (tecla 7)
T = 1/f	período	Duração de um ciclo.	852 Hz → 1,17 ms
φ	fase	Em que ponto do ciclo a onda começa.	0 (não importa para o DTMF)
ω = 2πf	frequência angular	A mesma coisa que f, em rad/s.	—

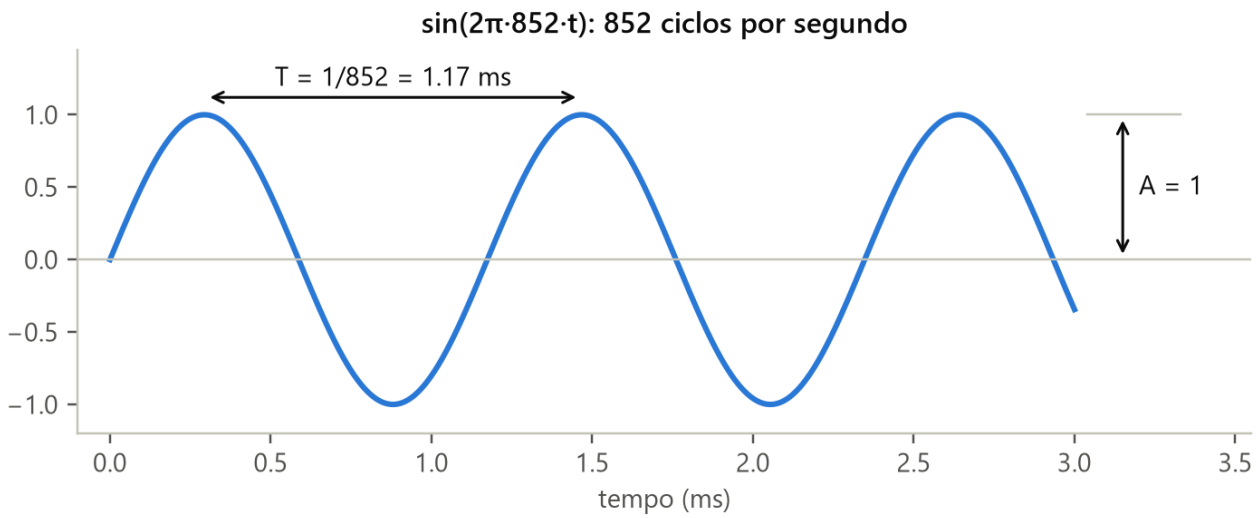


Figura 1 — Uma senoide de 852 Hz: amplitude A e período T = 1/f.

O ouvido humano escuta de ~20 Hz a ~20 kHz, e todas as frequências DTMF (697 a 1633 Hz) ficam na região em que a voz humana e a linha telefônica funcionam bem (300 a 3400 Hz). Um tom DTMF é a **soma** de duas senoides:

$$s(t) = \sin(2\pi f_{\text{baixa}}\, t) + \sin(2\pi f_{\text{alta}}\, t)$$

3.3 Digitalização: amostragem, quantização e codificação

Grandezas físicas são **contínuas**: em qualquer intervalo de tempo existem infinitos instantes, e em qualquer faixa de valores existem infinitos valores possíveis. O computador só guarda uma quantidade finita de números com um número finito de bits. Digitalizar é aceitar duas perdas:

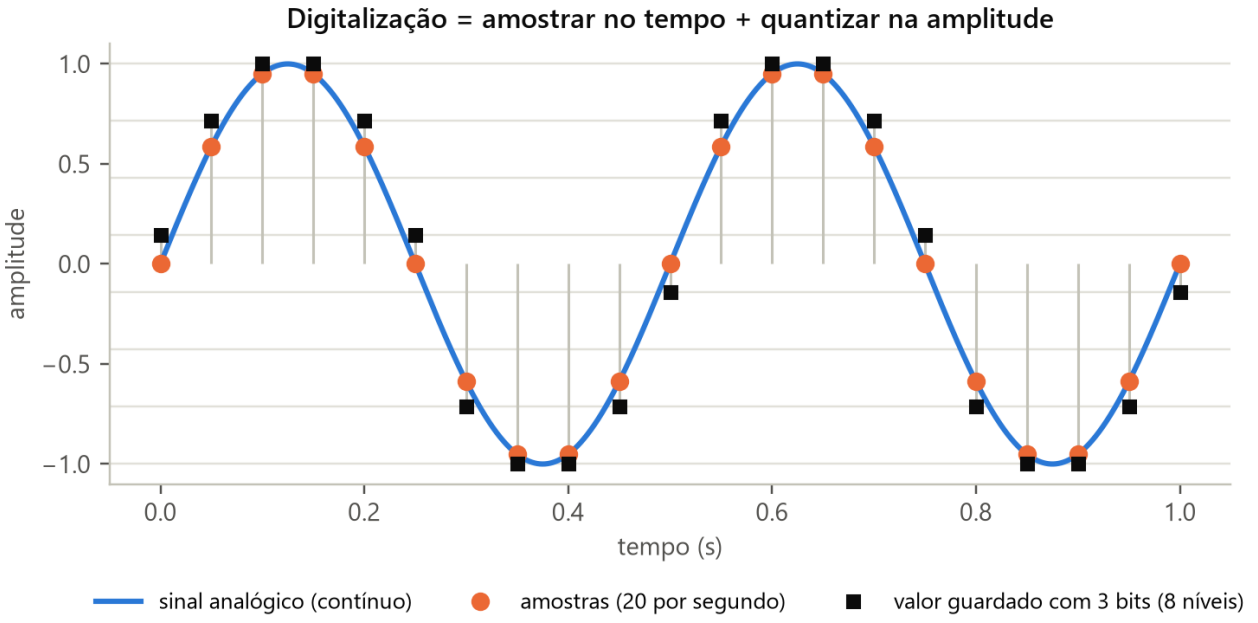


Figura 2 — Amostragem (bolinhas: o sinal medido a intervalos regulares) e quantização (quadrados: cada amostra arredondada para um dos 2^3 = 8 níveis de um conversor de 3 bits).

1. **Amostragem** (tempo discreto): mede-se o sinal a intervalos regulares. A **taxa (frequência) de amostragem** f_s é quantas amostras por segundo; o **período de amostragem** $T_s = 1/f_s$ é o tempo entre duas amostras. No projeto: f_s = 44 100 Hz, $T_s \approx$ 22,7 µs.

2. **Quantização** (amplitude discreta): cada amostra é arredondada para o nível mais próximo entre os 2^n níveis possíveis com n bits. O arredondamento é o **erro de quantização** (no máximo meio degrau, Δ/2).
3. **Codificação**: o nível escolhido vira um número binário de n bits. 8 bits → 256 níveis; 12 bits → 4096; 16 bits → 65 536.

$$\Delta = \frac{V_{max} - V_{min}}{2^n} \qquad \text{taxa de bits} = f_s \times n_{bits} \times n_{canais}$$

A **resolução** melhora com mais bits ou com uma faixa de entrada menor (os mesmos 2^n níveis espalhados num intervalo menor ficam mais próximos). Exemplo do CD: 44 100 × 16 × 2 = 1 411 200 bit/s ≈ 1,41 Mbit/s.

LIGAÇÃO COM O CÓDIGO

Isso aparece no próprio projeto. O `sd.play` recebe números reais entre −1 e 1 e a placa de som os quantiza (tipicamente com 16 ou 24 bits). Quando o emissor salva o `.wav` em 16 bits (multiplica por 32 767 e arredonda), o receptor, ao ler esse arquivo, encontra picos de ruído por volta de **−100 dB**: é o **ruído de quantização** de 16 bits, previsto pela regra SNR ≈ 6,02·n + 1,76 dB ≈ 98 dB.

Hardware A/D. O conversor mais rápido é o **flash**, mostrado na lista: um divisor resistivo cria $2^n - 1$ tensões de referência, cada uma ligada a um comparador (amplificador operacional). Quanto maior o sinal, mais comparadores "viram", e um codificador de prioridade converte quantos viraram num número binário. Com 3 bits são 7 comparadores; com 8 bits seriam 255, por isso o flash é rápido mas caro.

3.4 Nyquist e aliasing

Quantas amostras por segundo bastam? O **teorema de Nyquist–Shannon** responde: para representar uma frequência f, é preciso amostrar a **mais do que o dobro** dela. A metade da taxa de amostragem é a **frequência de Nyquist**, a maior frequência que a amostragem consegue representar.

$$f_s > 2 f_{max} \qquad f_{Nyquist} = \frac{f_s}{2}$$

- Áudio a 44 100 Hz representa até **22 050 Hz**: cobre a audição humana (~20 kHz) com folga para o filtro anti-aliasing. É por isso que CDs e placas de som usam 44,1 kHz (ou 48 kHz).
- Para DTMF bastaria $f_s > 2 \times 1633 =$ **3266 Hz**. A telefonia digital usa 8000 Hz (Nyquist = 4 kHz).
- Se uma frequência acima de $f_s/2$ entra no conversor, ela não some: aparece disfarçada de frequência mais baixa, falsa. Isso é o **aliasing**. A frequência falsa é $|f - k \cdot f_s|$ (o k inteiro que a deixa entre 0 e $f_s/2$). Por isso o ADC tem antes um filtro passa-baixas (anti-aliasing).

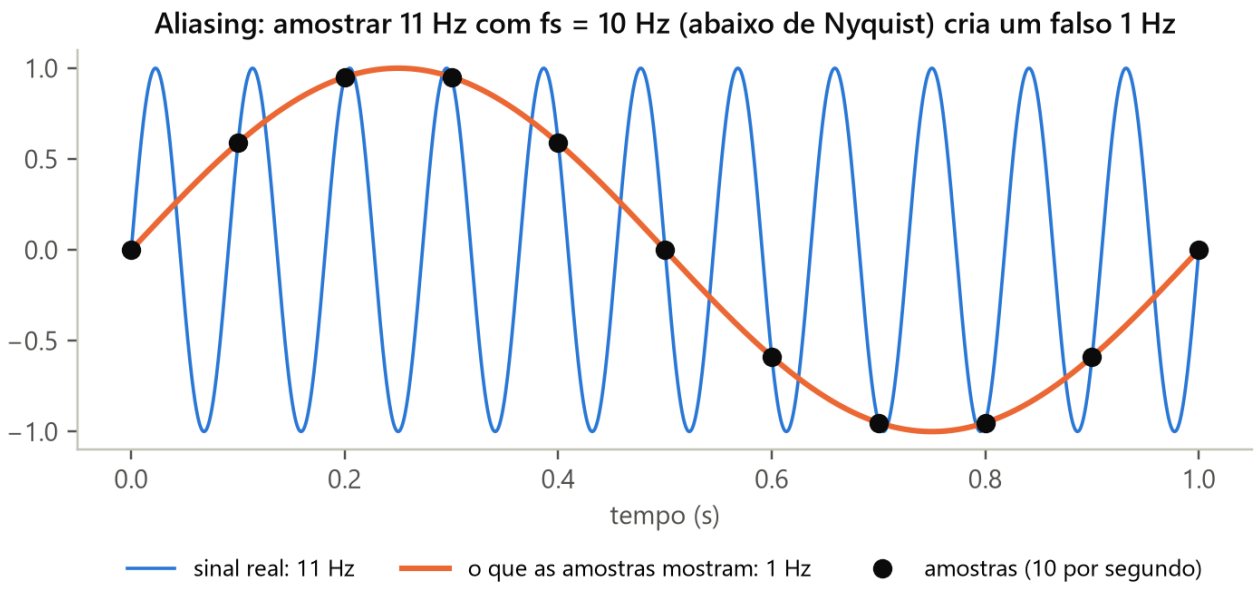


Figura 3 — Aliasing: amostrando 11 Hz a apenas 10 amostras por segundo, os pontos desenham uma senoide de $|11 - 10| = 1$ Hz. As amostras não têm como saber qual das duas era a real.

ANALOGIA

É o efeito da roda de carroça no cinema: a câmera tira 24 fotos por segundo (amostra); se a roda gira quase uma volta inteira entre duas fotos, ela parece girar devagar, ou até para trás.

3.5 Domínio do tempo × domínio da frequência

O mesmo sinal pode ser descrito de duas formas. No **domínio do tempo**, perguntamos "quanto vale o sinal em cada instante?" (é o que o microfone mede). No **domínio da frequência**, perguntamos "quais senoides, com que amplitudes, somadas, formam este sinal?". Para o DTMF a segunda pergunta é a que importa: o gráfico no tempo da tecla 7 é uma onda irregular, mas no domínio da frequência ele é só **dois picos**, em 852 e 1209 Hz.

ANALOGIA

Ouvir um acorde no piano: no tempo, a pressão do ar é uma onda complicada; mas o ouvido treinado diz "é um dó, um mi e um sol". O ouvido está fazendo uma análise de frequências. A Transformada de Fourier é esse ouvido em forma de conta. Outra imagem: o prisma que separa a luz branca nas cores (frequências) que a compõem.

3.6 Série e Transformada de Fourier

Joseph Fourier (1768–1830) mostrou que **qualquer sinal periódico é uma soma de senoides** (a série de Fourier). Até uma onda quadrada, cheia de quinas, sai somando senoides de frequências 1, 3, 5, 7... com amplitudes cada vez menores:

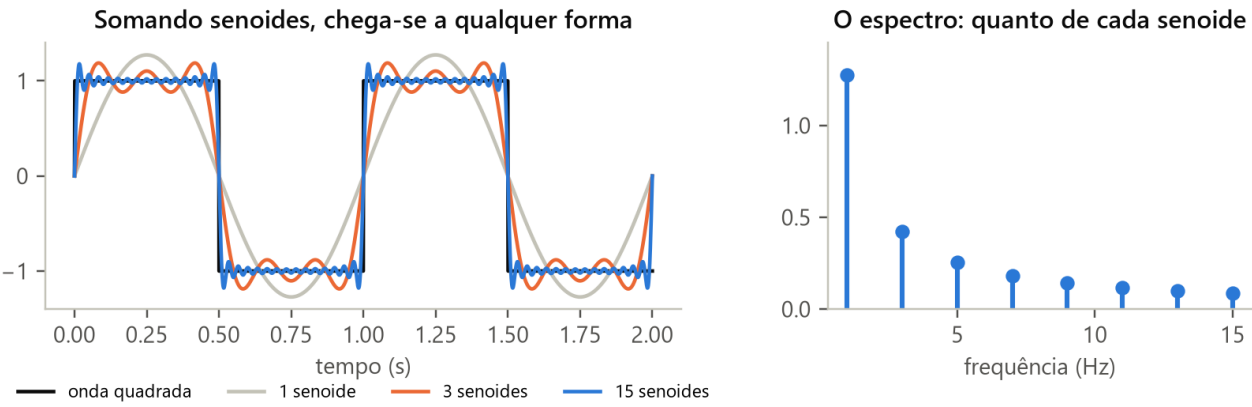


Figura 4 — Série de Fourier da onda quadrada. À direita, o espectro: a "receita" de quanto entra de cada frequência (amplitudes 4/(πk) para k ímpar).

A **Transformada de Fourier** generaliza a ideia para qualquer sinal e entrega a receita completa:

$$X(f) = \int_{-\infty}^{+\infty} x(t)\, e^{-j2\pi ft}\, dt \qquad e^{-j\theta} = \cos\theta - j\sin\theta$$

Intuição da conta: para saber quanto da frequência f existe no sinal, multiplica-se o sinal por uma senoide de teste de frequência f e soma-se (integra) o resultado. Se o sinal contém f, os dois sobem e descem juntos, o produto fica quase sempre positivo e a soma é grande ("ressonância"). Se não contém, o produto oscila entre positivo e negativo e a soma dá praticamente zero. A exponencial complexa testa cosseno e seno ao mesmo tempo (Euler), então a fase do sinal não atrapalha.

- O resultado X(f) é um **número complexo**: o **módulo |X(f)|** diz quanto de f existe; o **ângulo** diz a fase. Nos gráficos do projeto plotamos só o módulo.
- Para um sinal **real** (como o som), o espectro é simétrico: |X(−f)| = |X(f)|. Os slides mostram isso: sin(2π·5t) gera picos em +5 e −5 Hz. Por isso a `calcFFT` devolve só as frequências positivas, de 0 a $f_s/2$.
- Um nível constante (DC, "offset") aparece como um pico em 0 Hz. O receptor subtrai a média da gravação para eliminá-lo.

3.7 DFT e FFT: Fourier no computador

O computador não tem o sinal contínuo, e sim N amostras x[0], …, x[N−1]. A versão discreta da transformada é a **DFT**: entra uma lista de N números reais, sai uma lista de N números complexos (como dizem os slides).

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N} \qquad f_k = k\frac{f_s}{N} \qquad \Delta f = \frac{f_s}{N} = \frac{1}{T}$$

- Cada posição k da saída (um **bin**) corresponde à frequência $f_k = k \cdot f_s/N$.
- A distância entre dois bins é a **resolução em frequência** $\Delta f = f_s/N$. Como $N = f_s \cdot T$ (T = duração analisada), **Δf = 1/T**: só depende de **quanto tempo** se observa o sinal. Gravando 2 s, Δf = 0,5 Hz; gravando 20 ms, Δf = 50 Hz.
- A **FFT** (*Fast Fourier Transform*) é um algoritmo rápido que calcula exatamente a mesma DFT. A DFT direta custa ~N^2 operações; a FFT, ~$N \cdot \log_2 N$. Para N = 88 200 (2 s de áudio): 7,8 bilhões contra ~1,5 milhão, cerca de 5000 vezes mais rápido.
- A altura do pico é proporcional à amplitude e ao número de amostras: |X| ≈ A·N/2 × 0,54 (0,54 é o ganho da janela de Hamming). No emissor, A = 1 e N = 5 × 44 100 = 220 500 → |X| ≈ 59 500, exatamente a altura

dos picos na Figura 9c.

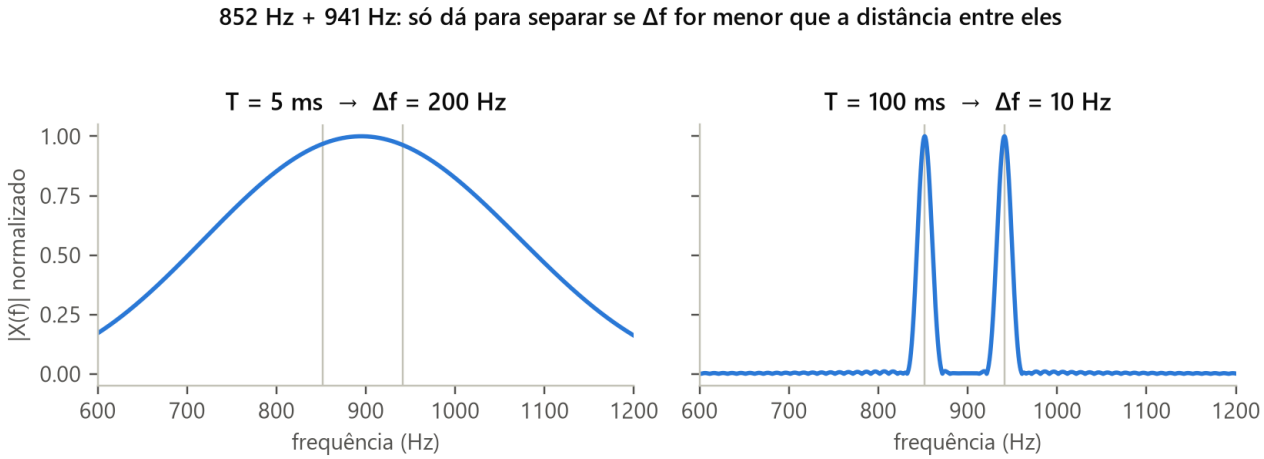


Figura 5 — Resolução. Com 5 ms de sinal (Δf = 200 Hz) os tons de 852 e 941 Hz viram um morro só; com 100 ms (Δf = 10 Hz) aparecem separados. É por isso que a norma DTMF exige tons de pelo menos ~40 ms.

PEGADINHA CLÁSSICA

Aumentar a taxa de amostragem **não** melhora a resolução: dobrar f_s dobra N, e $\Delta f = f_s/N$ fica igual. Mais f_s só aumenta a frequência máxima ($f_s/2$). Para enxergar detalhes finos de frequência é preciso observar por **mais tempo**.

3.8 Vazamento espectral e a janela de Hamming

A FFT trata o trecho analisado como se ele se repetisse para sempre. Se o trecho corta uma senoide no meio de um ciclo, a emenda entre o fim e o começo cria um salto, e esse salto espalha energia para frequências vizinhas que não existem no sinal: é o **vazamento espectral** (*leakage*), que aparece como "lóbulos laterais" em volta de cada pico.

A solução é multiplicar o trecho por uma **janela**, uma curva em forma de sino que vai suavemente a quase zero nas pontas, eliminando o salto. A `calcFFT` usa a janela de **Hamming**:

$$w[n] = 0{,}54 - 0{,}46\cos\left(\frac{2\pi n}{N-1}\right), \qquad n = 0, 1, \ldots, N-1$$

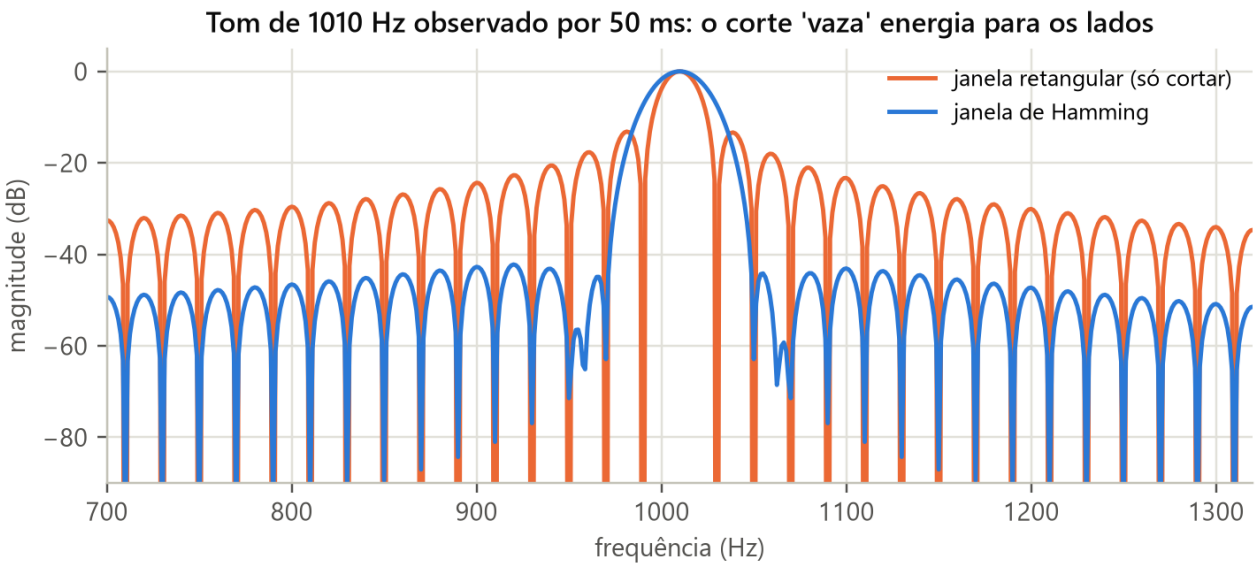


Figura 6 — Sem janela (laranja) os lóbulos laterais ficam só ~13 dB abaixo do pico; com Hamming (azul) caem para ~43 dB abaixo. O preço: o pico principal fica mais largo.

A troca é sempre esta: **menos vazamento, pico um pouco mais largo** (resolução levemente pior) e altura multiplicada por 0,54. Para o DTMF, com 2 s de gravação, a largura extra é irrelevante, e o vazamento menor evita que um tom forte esconda o outro.

3.9 Detecção de picos

Um **pico** é um máximo local: um ponto maior que os vizinhos (y[k−1] < y[k] > y[k+1]). Num espectro real, o ruído cria milhares de máximos locais minúsculos, então um detector precisa de dois critérios extras:

- **Altura mínima**. Em `peakutils.indexes(y, thres, min_dist)`, `thres` é uma **fração de 0 a 1** da faixa (máximo − mínimo): `thres=0.3` só aceita picos acima de 30% do maior. Quanto menor o thres, **mais sensível** o detector e mais picos aparecem.
- **Distância mínima**. `min_dist` é medido em **amostras do vetor** (bins), não em Hz. Para impor 5 Hz: min_dist = 5 / Δf (com Δf = 0,5 Hz, são 10 bins). Dentro dessa distância, fica só o pico mais alto.
- O equivalente no SciPy é `scipy.signal.find_peaks(y, height=..., distance=...)`, que o receptor usa automaticamente se o peakutils não estiver instalado.

3.10 DTMF: a ideia e o porquê das frequências

O DTMF foi lançado pela Bell System em 1963 com o nome *Touch-Tone*, substituindo a discagem por pulsos do telefone de disco. É padronizado pela ITU-T nas recomendações **Q.23** (frequências) e **Q.24** (recepção). Cada tecla é a soma de **uma frequência do grupo baixo** (a linha) com **uma do grupo alto** (a coluna): com só 8 frequências fazem-se 4 × 4 = 16 teclas.

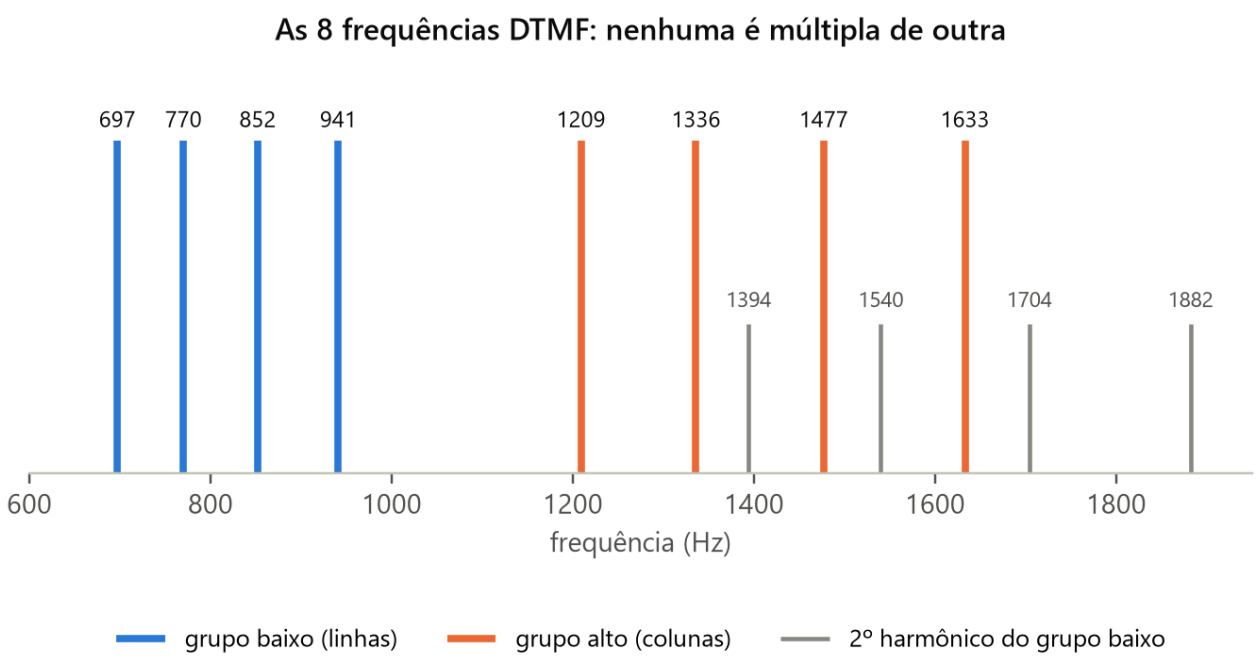


Figura 7 — As 8 frequências e os 2os harmônicos do grupo baixo. Os harmônicos caem entre as colunas, nunca em cima de uma.

Escolha de projeto	Motivo
Dois tons por tecla, não um	A voz ou a música podem conter um tom puro por acaso e "discar" sem querer (*talk-off*). Conter exatamente um tom de cada grupo, estáveis e simultâneos, é muito improvável. E 4 + 4 frequências geram 16 símbolos.
Frequências entre 697 e 1633 Hz	Dentro da banda que a linha telefônica transmite bem (300–3400 Hz).
Nenhuma é múltipla de outra	Distorção gera harmônicos (2f, 3f…). Se 2 × uma linha caísse sobre uma coluna, a distorção criaria uma tecla errada. Ex.: 2 × 697 = 1394 Hz fica entre 1336 e 1477.
Vizinhas separadas por ~10,5%	Razão ≈ 1,105 entre frequências consecutivas de cada grupo: espaço de sobra para tolerância de ±1,5%.
Grupos bem separados	Há um vão de 941 a 1209 Hz, e o detector sabe que precisa de exatamente um tom em cada lado.

- **Tolerâncias típicas da norma**: o receptor deve aceitar desvios de até ±1,5% na frequência e rejeitar acima de ±3,5% (o código usa 2%); o tom deve durar pelo menos ~40 ms, com pausa de ~40 ms entre teclas.
- **Twist**: diferença de volume entre os dois tons. Costuma-se mandar o grupo alto um pouco mais forte, porque a linha atenua mais as frequências altas. Por isso os dois picos no receptor raramente têm a mesma altura.
- As teclas **A, B, C, D** (coluna de 1633 Hz) existiam em redes militares e de controle; telefones comuns não as têm.

4 A solução: visão geral

Três arquivos, um caminho de ponta a ponta

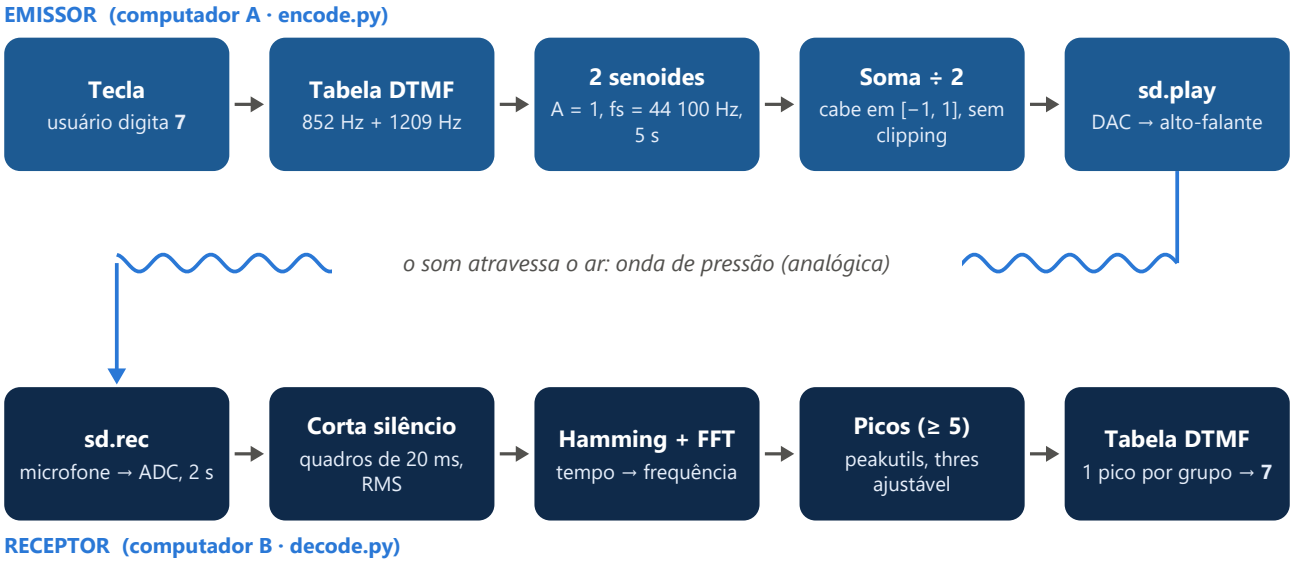


Figura 8 — O caminho completo. Em cima, o que roda no computador A; embaixo, no computador B.

Arquivo	Responsabilidade
suaBibSignal.py	Tabela DTMF (tecla ↔ par de frequências) e a classe `signalMeu`: `generateSin` (senoide amostrada), `calcFFT` (Hamming + FFT, só frequências positivas) e `plotFFT`.
encode.py	Pergunta a tecla, gera e soma as senoides, normaliza, toca, plota (tempo e Fourier) e opcionalmente salva o .wav.
decode.py	Grava (ou lê um .wav), separa o canal, corta silêncio, faz a FFT, detecta ≥ 5 picos, identifica a tecla, imprime e plota.

Parâmetros e por que cada valor

Parâmetro	Valor	Por quê
FS	44 100 Hz	Pedido nos comentários do modelo; padrão das placas de som; Nyquist 22 050 Hz, muito acima de 1633 Hz.
DURACAO (emissor)	5 s	"Alguns segundos", tempo de sobra para o receptor gravar.
AMPLITUDE	1	Pedido no modelo. A soma é dividida por 2 antes de tocar (seção 5).
DURACAO (receptor)	2 s	Δf = 0,5 Hz, muito mais fino que o necessário, e cabe dentro do tom de 5 s.
ESPERA	3 s	Contagem regressiva para disparar o emissor; assim a gravação começa com o som já tocando.
F_MIN, F_MAX	500–2000 Hz	Contém 697–1633 Hz e exclui o zumbido de 60 Hz e seus harmônicos, ventoinhas e voz grave.
MIN_PICOS / MAX_PICOS	5 / 10	O enunciado exige ≥ 5; mostrar mais de 10 só polui.
DIST_PICOS_HZ	5 Hz	Sugestão do modelo: picos a menos de 5 Hz um do outro são o mesmo pico.
TOLERANCIA	2%	Entre o "aceita ±1,5%" e o "rejeita ±3,5%" da norma; vizinhas estão a ~10%.

Parâmetro	Valor	Por quê
FATOR_PISO	5× (14 dB)	Um pico só vale se for 5× maior que a mediana do espectro: evita "achar" tecla em silêncio.

5 Emissor passo a passo

encode.py: da tecla ao som saindo do alto-falante

Passo 1 — tecla e frequências. O programa pergunta a tecla (em laço até ser válida) e consulta `TABELA_DTMF`, montada a partir das listas de linhas e colunas:

```
FREQS_BAIXAS = (697, 770, 852, 941)        # linhas do teclado
FREQS_ALTAS = (1209, 1336, 1477, 1633)     # colunas do teclado
...
TABELA_DTMF["7"]   # -> (852, 1209)
```

Passo 2 — gerar as senoides. A base é o vetor de tempo: um instante para cada amostra, separados por $1/f_s$. Com 5 s a 44 100 Hz são 220 500 instantes. Aplicando $A \cdot \sin(2\pi f t)$ em cada um, temos a senoide amostrada:

```
def generateSin(self, freq, amplitude, time, fs):
    """Senoide A*sin(2*pi*f*t) com `time` segundos amostrada a `fs` Hz."""
    n = int(round(time * fs))
    t = np.arange(n) / fs
    s = amplitude * np.sin(2 * np.pi * freq * t)
    return (t, s)
```

```
def gerar_tom(tecla, duracao=DURACAO, fs=FS):
    """Gera o tom DTMF da tecla. Retorna (t, s_baixa, s_alta, tom)."""
    signal = signalMeu()
    f_baixa, f_alta = TABELA_DTMF[tecla]
    t, s_baixa = signal.generateSin(f_baixa, AMPLITUDE, duracao, fs)
    _, s_alta = signal.generateSin(f_alta, AMPLITUDE, duracao, fs)
    tom = s_baixa + s_alta
    return t, s_baixa, s_alta, tom
```

Passo 3 — normalizar. Cada senoide vai de −1 a 1, então a soma vai de **−2 a 2**. A placa de som só aceita valores entre −1 e 1: o que passa disso é cortado (**clipping**), a onda fica achatada no topo, e uma onda achatada não é mais a soma de duas senoides puras. Aparecem harmônicos e produtos de intermodulação ($2 \cdot 852 - 1209 = 495$ Hz, $2 \cdot 1209 - 852 = 1566$ Hz…), que o receptor veria como picos falsos. Dividir por 2 encolhe a onda sem mudar a forma.

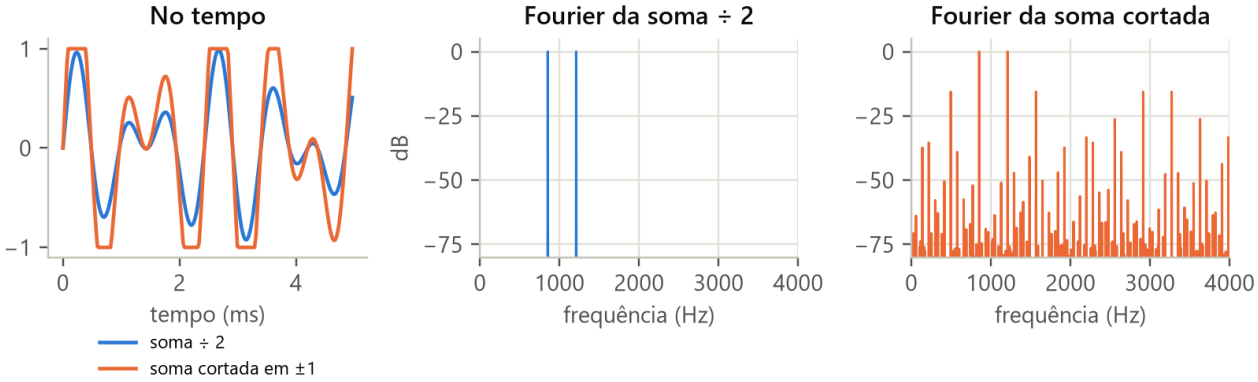


Figura 9a — A mesma soma, dividida por 2 (azul) e cortada em ±1 (laranja). O corte cria dezenas de picos falsos no espectro, alguns só 15 dB abaixo dos verdadeiros.

Passo 4 — tocar e plotar. `sd.play` **não bloqueia**: entrega o vetor à placa de som e retorna na hora, então os gráficos são montados com o som tocando. `plt.show()` segura o programa até as janelas serem fechadas, e `sd.wait()` garante que o áudio terminou.

```
def emitir(tecla, duracao, salvar):
    f_baixa, f_alta = TABELA_DTMF[tecla]
    print("Gerando Tons base")
    t, s_baixa, s_alta, tom = gerar_tom(tecla, duracao)

    # Cada senoide tem amplitude 1, então a soma vai de -2 a 2. A placa de som
    # só aceita valores entre -1 e 1: acima disso o sinal é cortado (clipping),
    # o que distorce a onda e cria frequências que não existiam. Dividir por 2
    # mantém o formato da onda e cabe na faixa.
    tom_saida = tom / 2

    print(f"Gerando Tom referente ao símbolo : {tecla}")
    print(f"Frequências : {f_baixa} + {f_alta} Hz")
    if salvar:
        salvar_wav(tecla, tom_saida)

    print(f"Executando as senoides (emitindo o som por {duracao} s)")
    sd.play(tom_saida, FS)      # não bloqueia: o som toca enquanto plotamos

    print("Plotando")
    plotar(tecla, t, s_baixa, s_alta, tom)
    plt.show()                  # bloqueia até as janelas serem fechadas

    sd.wait()                   # garante que o áudio terminou
```

Passo 5 — o que o programa mostra. Saída no terminal e os dois gráficos pedidos:

```
Inicializando encoder
Aguardando usuário
Digite a tecla (0-9, A-D, *, #) ou q para sair: 7
Gerando Tons base
Gerando Tom referente ao símbolo : 7
Frequências : 852 + 1209 Hz
Executando as senoides (emitindo o som por 5 s)
Plotando
```

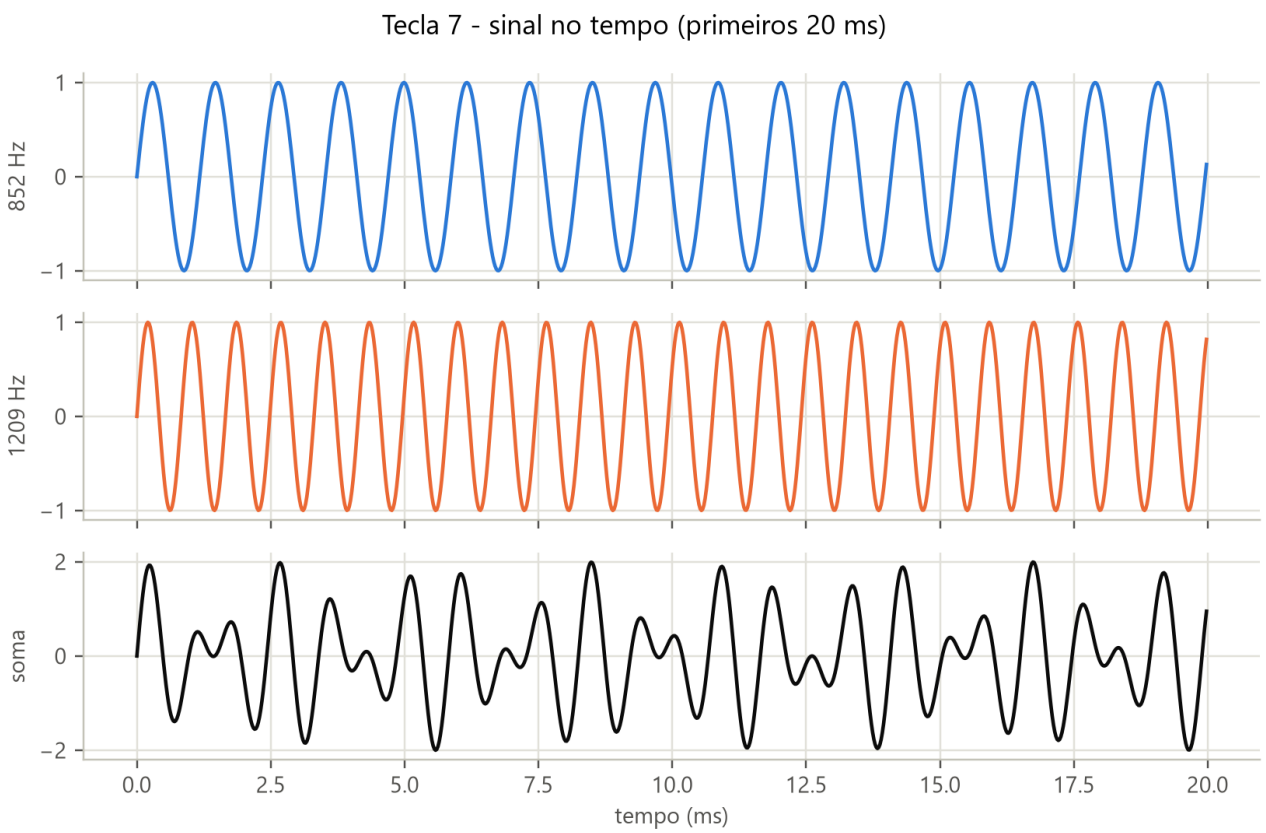


Figura 9b — Gráfico do tempo do emissor (tecla 7): as duas senoides e a soma, só nos primeiros 20 ms. Com os 5 s inteiros (220 500 pontos, 4260 ciclos de 852 Hz) o gráfico viraria uma mancha.

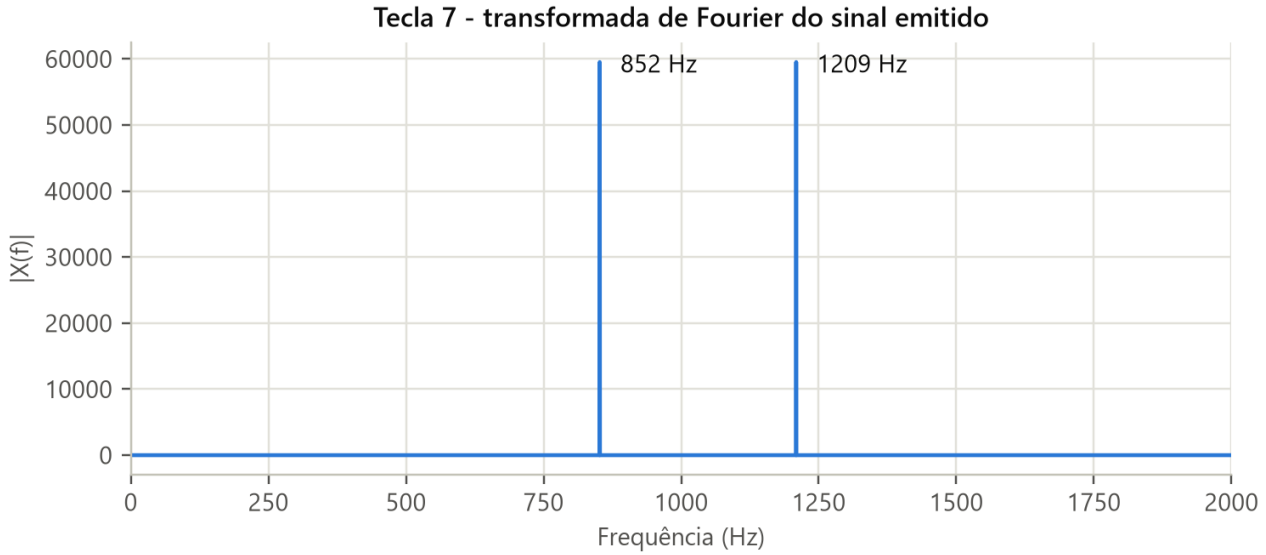


Figura 9c — Fourier do sinal emitido: dois picos finos, exatamente em 852 e 1209 Hz, da mesma altura (mesma amplitude), e nada mais. Altura ≈ A·N/2 × 0,54 ≈ 59 500.

Passo 6 (opcional) — salvar. Com `--salvar`, o tom normalizado é convertido para inteiros de 16 bits (× 32 767) e gravado em `tons/tecla_7.wav`. 5 s ocupam 220 500 × 2 bytes ≈ 441 kB. O receptor decodifica esse arquivo com `--arquivo`, ótimo para testar sem microfone.

6 Receptor passo a passo

decode.py: do microfone à tecla impressa na tela

Passo 1 — gravar. Configura o sounddevice, calcula quantas amostras gravar (numAmostras = duração × f_s = 2 × 44 100 = 88 200), faz a contagem regressiva para o emissor ser disparado e grava. Assim como `sd.play`, `sd.rec` não bloqueia; `sd.wait()` espera a gravação acabar.

```
def gravar(duracao=DURACAO, espera=ESPERA, fs=FS):
    sd.default.samplerate = fs
    sd.default.channels = CANAIS
    numAmostras = int(duracao * fs)

    print(f"A captação começará em {espera} segundos. Dispare o emissor!")
    for falta in range(espera, 0, -1):
        print(f"  {falta}...")
        time.sleep(1)

    print("Gravação inicializada")
    audio = sd.rec(int(numAmostras), fs, channels=CANAIS)
    sd.wait()
    print("...     FIM")
    return audio
```

Passo 2 — extrair os dados. `sd.rec` devolve uma **matriz** (amostras × canais): com 1 canal, o formato é (88 200, 1), uma "lista de listas", como o modelo avisa. Ficamos com a coluna 0 e subtraímos a média (o nível DC do microfone, que viraria um pico em 0 Hz).

```
def extrair_dados(audio):
    """A gravação pode vir como matriz (amostras x canais). Fica com um canal."""
    dados = np.asarray(audio, dtype=float)
    if dados.ndim > 1:
        dados = dados[:, 0]
    return dados - np.mean(dados)       # tira o nível DC do microfone
```

Passo 3 — cortar o silêncio. O enunciado pede para não gravar silêncio. Além de começar a gravar com o som já tocando, o receptor divide a gravação em quadros de 20 ms, mede a energia (RMS) de cada um e mantém só do primeiro ao último quadro com energia acima de 30% da maior. Silêncio só adiciona ruído à FFT.

```
def trecho_com_som(dados, fs, quadro=0.02, limiar=0.3):
    """Índices (ini, fim) do trecho com som, descartando silêncio nas pontas.

    Divide a gravação em quadros de 20 ms e mantém do primeiro ao último
    quadro cuja energia (RMS) passa de 30% da maior energia.
    """
    n = int(quadro * fs)
    nq = len(dados) // n
    if nq < 2:
        return 0, len(dados)
    rms = np.sqrt(np.mean(dados[:nq * n].reshape(nq, n) ** 2, axis=1))
    ativos = np.nonzero(rms >= limiar * rms.max())[0]
    return ativos[0] * n, (ativos[-1] + 1) * n
```

Passo 4 — Fourier. `calcFFT` aplica Hamming, faz a FFT e devolve frequências (xf) e módulos (yf) de 0 a 22 050 Hz:

```
def calcFFT(self, signal, fs):
    """Módulo da FFT (só frequências positivas) do sinal janelado.

    Retorna (xf, yf): xf são as frequências em Hz (passo fs/N) e yf o
    módulo |X(f)| correspondente. A janela de Hamming reduz o vazamento
    espectral causado por cortar o sinal num trecho finito.
    """
    signal = np.asarray(signal, dtype=float)
    N = len(signal)
    W = np.hamming(N)
    yf = np.fft.rfft(signal * W)
    xf = np.fft.rfftfreq(N, 1 / fs)
    return (xf, np.abs(yf))
```

Passo 5 — achar os picos. Só na faixa 500–2000 Hz. A sensibilidade começa exigente (`thres=0.5`) e vai sendo afrouxada até aparecerem pelo menos 5 picos, que é o "ajuste a sensibilidade para ter no mínimo 5 picos" do enunciado. O último degrau (`thres=0`) aceita qualquer máximo local; só é necessário quando o sinal não tem ruído nenhum, como um .wav gerado no computador.

```
def detectar_picos(xf, yf, fmin=F_MIN, fmax=F_MAX, min_picos=MIN_PICOS):
    """Picos de |X(f)| na faixa [fmin, fmax], do mais forte para o mais fraco.

    `thres` é a altura mínima do pico como fração do maior valor (0 a 1).
    Começamos exigentes e baixamos até aparecerem pelo menos `min_picos`.
    """
    faixa = (xf >= fmin) & (xf <= fmax)
    xb, yb = xf[faixa], yf[faixa]
    df = xf[1] - xf[0]
    min_dist = max(1, int(round(DIST_PICOS_HZ / df)))

    for thres in LIMIARES:
        idx = np.asarray(indices_de_picos(yb, thres, min_dist), dtype=int)
        if len(idx) >= min_picos:
            break
    idx = idx[np.argsort(yb[idx])[::-1]][:MAX_PICOS]
    piso = np.median(yb)
    return xb[idx], yb[idx], piso, thres
```

Passo 6 — decidir a tecla. Percorre os picos do mais forte para o mais fraco e guarda o primeiro que estiver a até 2% de uma frequência do **grupo baixo** e o primeiro perto de uma do **grupo alto**, ignorando picos fracos demais (abaixo de 5× a mediana do espectro). O par (baixa, alta) é procurado na tabela invertida `TECLA_POR_FREQS`.

```
def mais_proxima(f, grupo):
    """Frequência DTMF do grupo mais próxima de f, ou None se longe demais."""
    alvo = min(grupo, key=lambda g: abs(g - f))
    return alvo if abs(alvo - f) <= TOLERANCIA * alvo else None

def identificar_tecla(freqs, mags, piso):
    """Escolhe o pico mais forte de cada grupo (baixo e alto) e acha a tecla.

    Toda tecla é exatamente uma frequência baixa (linha) + uma alta (coluna),
    então nunca escolhemos dois picos do mesmo grupo.
    """
    baixa = alta = None
    for f, m in zip(freqs, mags):           # já vem do mais forte ao mais fraco
        if m < FATOR_PISO * piso:
            continue                        # pico de ruído, fraco demais
        if baixa is None and mais_proxima(f, FREQS_BAIXAS):
            baixa = (mais_proxima(f, FREQS_BAIXAS), f, m)
        if alta is None and mais_proxima(f, FREQS_ALTAS):
            alta = (mais_proxima(f, FREQS_ALTAS), f, m)
    tecla = None
    if baixa and alta:
        tecla = TECLA_POR_FREQS[(baixa[0], alta[0])]
    return tecla, baixa, alta
```

O "CUIDADO!" DO ENUNCIADO

O enunciado avisa "Cuidado!" na hora de identificar a tecla. Estas são as armadilhas e como o código trata cada uma:

- **Não pegue simplesmente os dois maiores picos.** Um ruído forte (ou um pico vizinho de um dos tons) pode ser o 2º maior. A tecla é sempre um tom de **cada** grupo, então escolhemos o mais forte de cada grupo.
- **Picos duplicados.** Eco, interferência e vazamento criam picos colados no verdadeiro, poucos Hz ao lado e bem mais fracos (na saída da seção 7: 1217,9 Hz perto de 1209 e 866 Hz perto de 852). O `min_dist` de 5 Hz junta os muito próximos, e a regra "mais forte do grupo" descarta o resto.
- **A frequência medida não é exata**: 851,9 Hz em vez de 852, por causa da resolução (bins), da diferença entre os relógios das duas placas de som (~0,1%) e do ruído. Daí a tolerância de 2%.
- **Harmônicos e intermodulação** (alto-falante saturado, volume no máximo) criam picos fora da tabela, ignorados pela tolerância. Os harmônicos da rede (60, 120, 180 Hz) ficam fora da faixa 500–2000 Hz.
- **Silêncio ou só ruído**: sem picos 14 dB acima do piso, a resposta é "nenhuma tecla" em vez de um chute.
- **min_dist é em bins, não em Hz**: com 2 s de gravação, 5 Hz = 10 bins; com 1 s, 5 bins.

Passo 7 — imprimir e plotar. A lista de picos com a altura em dB relativa ao maior, as duas frequências escolhidas e a tecla; depois os gráficos do tempo e de Fourier (seção 7).

7 Resultados e testes

O que o receptor mostra e como a solução foi validada

Para gerar os gráficos e testar sem tocar som em voz alta, o sinal do emissor passou por um **canal acústico simulado**: atenuação de 20×, eco de sala (resposta ao impulso de 60 ms), zumbido da rede elétrica (60 e 180 Hz), ruído branco, 0,4 s de silêncio antes do tom e diferença de relógio de até ±0,2% entre as placas. Abaixo, a tecla 7 recebida com SNR de 12 dB:

```
A captação começará em 3 segundos. Dispare o emissor!
  3...
  2...
  1...
Gravação inicializada
...     FIM

Picos encontrados (sensibilidade thres=0.002), do mais forte para o mais fraco:
   freq de pico =   1209.0 Hz   (    0.0 dB)
   freq de pico =    851.9 Hz   (   -2.7 dB)
   freq de pico =   1217.9 Hz   (  -49.1 dB)
   freq de pico =    866.0 Hz   (  -51.0 dB)
   freq de pico =   1843.6 Hz   (  -51.1 dB)
   freq de pico =   1640.4 Hz   (  -51.5 dB)
   freq de pico =   1939.1 Hz   (  -51.7 dB)
   freq de pico =    508.3 Hz   (  -51.9 dB)
   freq de pico =   1710.9 Hz   (  -51.9 dB)
   freq de pico =   1397.4 Hz   (  -52.1 dB)
Frequência baixa: 852 Hz (pico em 851.9 Hz)
Frequência alta : 1209 Hz (pico em 1209.0 Hz)

==> TECLA IDENTIFICADA: 7
```

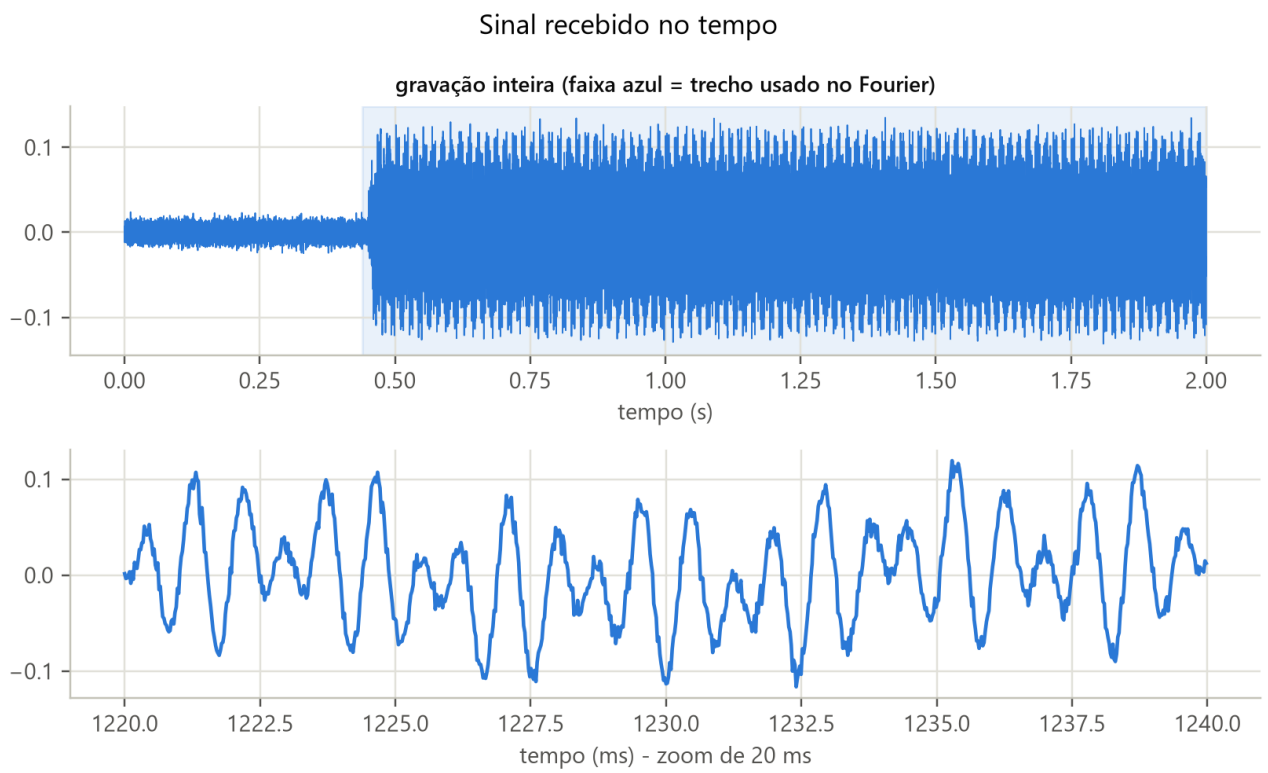


Figura 10 — Sinal recebido no tempo. Em cima, a gravação inteira: o silêncio (só ruído) até ~0,45 s e o tom depois; a faixa azul é o trecho enviado à FFT. Embaixo, um zoom de 20 ms: o mesmo desenho da soma do emissor, só que pequeno (atenuado pelo ar) e com ruído.

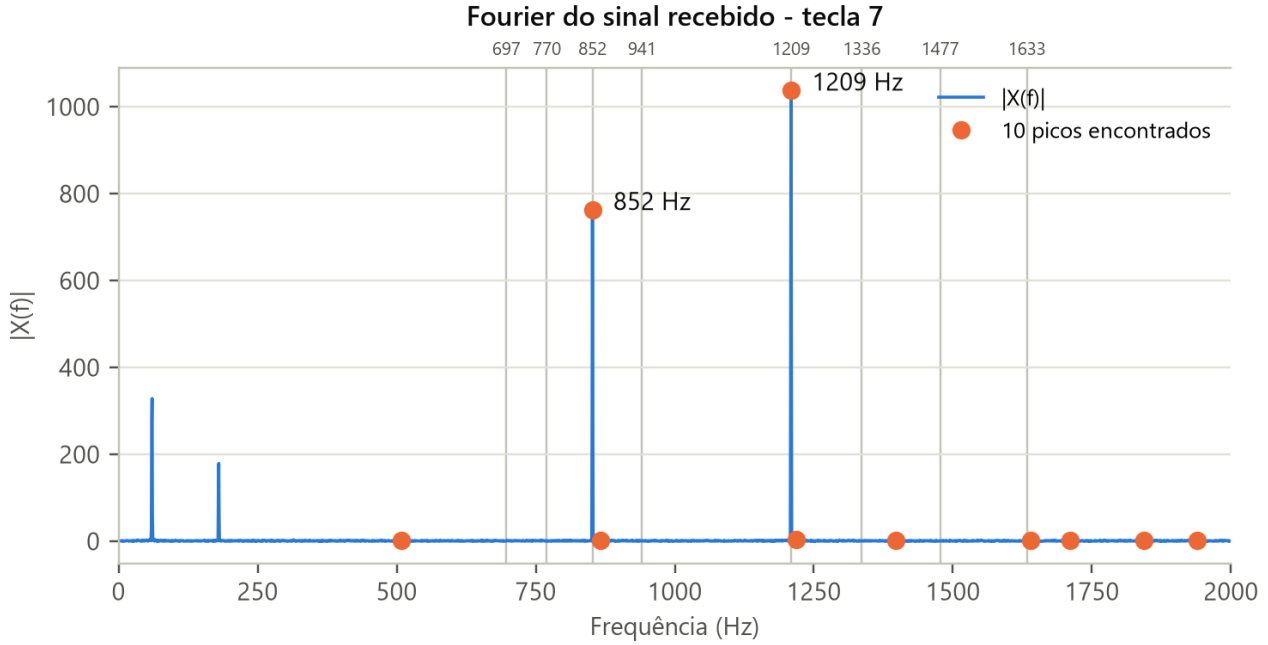


Figura 11 — Fourier do sinal recebido. Dois picos dominantes nas linhas DTMF de 852 e 1209 Hz; os outros 8 picos marcados são ruído, ~50 dB mais fracos. Os picos em 60 e 180 Hz (rede elétrica) ficam fora da faixa de busca e são ignorados.

Bateria de testes

Cenário	Acertos
Sinal limpo, direto do emissor (16 teclas)	16/16
Canal simulado, SNR 20 dB (16 teclas × 3)	48/48

Cenário	Acertos
Canal simulado, SNR 10 dB	48/48
Canal simulado, SNR 0 dB (ruído tão forte quanto o tom)	48/48
Canal simulado, SNR −10 dB (ruído 10× mais forte)	48/48
Canal simulado, SNR −20 dB (ruído 100× mais forte)	35/48
Canal simulado, SNR −30 dB (ruído 1000× mais forte)	6/48
Só ruído, sem tom: teclas inventadas	0 em 30
Receptor amostrando a 48 kHz em vez de 44,1 kHz	16/16
Arquivo .wav: encode --salvar → decode --arquivo (7, *, #)	3/3
Detector do SciPy (find_peaks) no lugar do peakutils	16/16

Em todos os cenários com tom o detector achou pelo menos 5 picos. O acerto de 100% mesmo com o ruído 10 vezes mais forte que o tom tem uma explicação: a FFT **concentra** toda a energia de uma senoide em um único bin, enquanto o ruído branco se espalha igualmente pelos ~44 000 bins. Esse "ganho de processamento" vale cerca de $10 \cdot \log_{10}(N/2) \approx 46$ dB.

LIMITE DO QUE FOI TESTADO

Os testes acima usam um canal **simulado**. A biblioteca sounddevice foi verificada nesta máquina (ela lista o microfone e as saídas de áudio), mas o teste acústico de verdade, com dois computadores ou com um celular, precisa ser feito no laboratório antes da apresentação. A seção 9 traz o roteiro e o que fazer se algo der errado.

8 Como interpretar os gráficos

O terceiro critério de avaliação: saber explicar o que se vê

Gráfico do tempo do emissor (Figura 9b)

- **Eixo x em ms**, só 20 ms: em 5 s há 220 500 pontos e milhares de ciclos; tudo junto vira uma mancha.
- As duas senoides de cima são regulares: 852 Hz completa ~17 ciclos em 20 ms; 1209 Hz, ~24.
- A soma (embaixo) **não é uma senoide**: os picos ora se somam (chega perto de 2, quando as duas estão no máximo juntas), ora se cancelam. É a "cara" de um sinal com duas frequências. Olhando só para ela, é difícil dizer quais são as frequências: por isso existe o gráfico de Fourier.
- A amplitude vai até 2 antes da divisão; o que é tocado vai até 1.

Gráfico de Fourier do emissor (Figura 9c)

- **Eixo x: frequência** (Hz). Só a parte positiva, porque o espectro de um sinal real é simétrico. Vai até $f_s/2$ = 22 050 Hz, mas o gráfico mostra 0–2000 Hz, onde está tudo.
- **Eixo y: |X(f)|**, quanto de cada frequência existe no sinal. É proporcional à amplitude (e ao número de amostras), então a unidade é arbitrária: o que importa é a comparação entre picos.
- Exatamente **dois picos**, em 852 e 1209 Hz, de mesma altura (as duas senoides têm A = 1). Fora deles, zero: o sinal emitido é matematicamente puro.

Gráfico do tempo do receptor (Figura 10)

- A amplitude é **bem menor** que a emitida: o som perde energia no ar e o microfone capta só uma parte.
- Antes do tom há só ruído (silêncio de verdade nunca é zero); a faixa azul mostra o trecho usado na FFT, sem o silêncio.
- No zoom aparece o mesmo padrão da soma, deformado por ruído e eco. Mesmo assim a FFT encontra os tons.

Gráfico de Fourier do receptor (Figura 11)

- Dois picos dominantes sobre as **linhas cinza** (as 8 frequências DTMF, rotuladas no topo): a linha de 852 Hz diz a **linha** do teclado e a de 1209 Hz, a **coluna** → tecla 7.
- As **bolinhas laranja** são os picos que o detector achou (no mínimo 5, como pede o enunciado). Só dois estão perto de frequências DTMF; os outros são ruído.
- Os picos não têm a mesma altura: alto-falante e microfone não respondem igual a todas as frequências (e há o *twist*). Por isso não se exige altura igual.
- Picos em **60 Hz e 180 Hz** são interferência da rede elétrica (60 Hz no Brasil) e harmônico. Ficam fora da faixa de busca de propósito.

TREINO DE APRESENTAÇÃO

Se o professor apontar para um pico e perguntar "o que é isto?", as respostas possíveis são:

- **Tom DTMF**: está sobre uma das 8 frequências da tabela e muito acima do resto.
- **Rede elétrica**: 60 Hz ou múltiplos (120, 180…).
- **Harmônico / intermodulação**: múltiplo de um tom, ou combinação como $2f_1 - f_2$ (aparece se o volume saturar).
- **Lóbulo / pico vizinho**: colado num pico grande, poucos Hz ao lado e dezenas de dB abaixo.
- **Ruído**: baixo, espalhado, sem posição especial.

9 Como rodar e roteiro de apresentação

Instalação, uso em dois computadores e solução de problemas

Instalação (nos dois computadores)

```
python -m pip install numpy scipy matplotlib sounddevice peakutils
```

Copie a pasta inteira (precisa de `suaBibSignal.py` junto com `encode.py` e `decode.py`).

Comandos

Comando	O que faz
`python encode.py`	Pergunta a tecla, toca 5 s e plota; repete até digitar q.
`python encode.py 7`	Toca a tecla 7 direto.
`python encode.py 7 --salvar --duracao 8`	Toca 8 s e salva tons/tecla_7.wav.
`python decode.py`	Conta 3 s, grava 2 s, identifica a tecla e plota.
`python decode.py --espera 5 --duracao 3`	Mais tempo para disparar o emissor; grava 3 s.

Comando	O que faz
`python decode.py --arquivo tons/tecla_7.wav`	Decodifica um .wav (gravado do celular, por exemplo) sem usar o microfone.

Roteiro de demonstração

1. No computador A, rode `python encode.py` e **digite a tecla, sem apertar Enter**.
2. No computador B, rode `python decode.py`. Começa a contagem 3, 2, 1.
3. Assim que a contagem começar, aperte Enter no A: o tom dura 5 s, então a gravação de 2 s cai inteira dentro dele.
4. B imprime os picos e a tecla e abre os dois gráficos; A abre os gráficos do sinal emitido.
5. Explique os quatro gráficos (seção 8) e repita com outras teclas: o critério é acertar **todas** as tentativas.
6. Bônus: grave a tecla de um telefone real e use `--arquivo`, ou toque o tom de um app de celular perto do microfone.

Quando algo dá errado

Sintoma	Causa provável e solução
Erro do PortAudio / nenhum dispositivo	Microfone desligado ou bloqueado. No Windows: Configurações → Privacidade → Microfone. Veja os dispositivos com `python -m sounddevice` e, se precisar, fixe um com `sd.default.device`.
"Nenhuma tecla DTMF reconhecida"	Gravou fora do tom (sincronize pelo roteiro, aumente `--duracao` no emissor) ou volume muito baixo/distância grande.
Tecla errada ou picos estranhos	Volume do alto-falante no máximo distorce (picos de intermodulação): abaixe um pouco. Afaste de ventiladores.
Só 2 ou 3 picos encontrados	Sinal sem ruído nenhum (um .wav sintético): o espectro só tem esses máximos. Com microfone sempre há ≥ 5.
"module 'scipy.signal' has no attribute 'hamming'"	Biblioteca de apoio antiga. Use a `suaBibSignal.py` desta pasta (np.hamming).
Gráficos não aparecem	Feche as janelas de um ciclo para o programa seguir; em alguns ambientes é preciso rodar fora do VS Code interativo.

10 Lista de exercícios — Digitalização

Resolvida passo a passo, como pede a entrega

Considerações do texto: amostragem a **44,1 kHz**, **2 canais** (estéreo) e **16 bits** por amostra.

Pergunta do texto — memória para 1 hora de áudio estéreo

Passo	Conta	Resultado
Bits por segundo, por canal	44 100 amostras/s × 16 bits	705 600 bit/s
Dois canais	705 600 × 2	1 411 200 bit/s
Uma hora	1 411 200 × 3600 s	5 080 320 000 bits
Em bytes	÷ 8	**635 040 000 bytes ≈ 635 MB**

Resposta: ≈ 635 MB (≈ 605,6 MiB, contando 1 MiB = 1 048 576 bytes). O CD de áudio usa exatamente este formato, e por isso comporta só ~74 a 80 minutos de música. Formatos como o MP3 comprimem para cerca de 10% disso.

Exercício 1 — ECG por um enlace de 1000 bps com 10% de overhead

Passo	Conta	Resultado
Duração	5 min × 60	300 s
Amostras	300 s × 1000 amostras/s	300 000 amostras
Bits de dados (payload)	300 000 × 14 bits	4 200 000 bits
Com overhead (total = 1,1 × payload)	4 200 000 × 1,1	4 620 000 bits
Tempo	4 620 000 bits ÷ 1000 bit/s	4620 s
Em minutos	4620 ÷ 60	**77 minutos**

Resposta: ≈ 77 minutos. Usamos a definição da aula de datagramas (Projeto 3): *overhead = tamanho total / payload*; overhead de 10% significa que o total é 1,1 × os dados. Se a questão entender que 10% da taxa é perdida (sobram 900 bit/s úteis), dá 4 200 000 ÷ 900 = 4667 s ≈ 77,8 min: os dois caminhos levam a **~77–78 min**.

INTERPRETAÇÃO

Repare na proporção: o ECG **gera** 14 000 bit/s (1000 amostras × 14 bits), mas o enlace transmite 1000 bit/s. Por isso 5 minutos de exame levam mais de uma hora para chegar: ~15 vezes a duração do sinal.

Exercício 2 — Bits para digitalizar o captador da guitarra

Passo	Conta	Resultado
Faixa do sinal	+50 mV − (−50 mV)	100 mV
Níveis necessários	100 mV ÷ 10 µV	10 000 níveis
Bits	$2^n \geq 10\,000 \rightarrow n \geq \log_2(10\,000) = 13{,}29$	**n = 14 bits**

Passo	Conta	Resultado
Conferência (13 bits)	100 mV ÷ 8192	12,2 µV — pior que 10 µV, não serve
Conferência (14 bits)	100 mV ÷ 16 384	6,1 µV — atende

Resposta: 14 bits por amostra. O número de bits é sempre arredondado **para cima**: 13,29 bits não existe, e 13 não chegam à resolução pedida.

Exercício 3 — Senoide de 4400 Hz gravada com Ts = 25 µs e tocada a 80 kHz

Passo	Conta	Resultado
Taxa de gravação	1 ÷ 25 µs	40 000 Hz
Nyquist na gravação	40 000 > 2 × 4400 = 8800	ok, sem aliasing
Amostras por ciclo	40 000 ÷ 4400	≈ 9,09 amostras
Duração de um ciclo na reprodução	9,09 ÷ 80 000 Hz	≈ 113,6 µs
Frequência ouvida	1 ÷ 113,6 µs (ou 4400 × 80 000 / 40 000)	**8800 Hz**

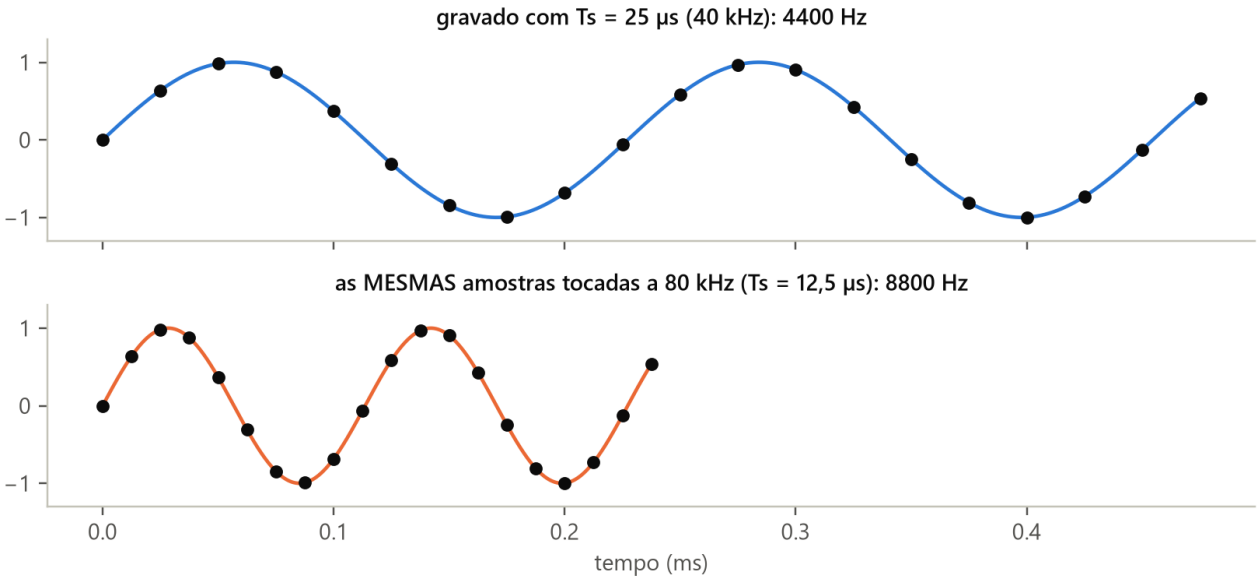


Figura 12 — As mesmas 20 amostras: gravadas a 40 kHz descrevem 4400 Hz; tocadas a 80 kHz, passam no dobro da velocidade e descrevem 8800 Hz.

Resposta: 8800 Hz. O arquivo digital não guarda a frequência, só a lista de amostras. Quem decide a frequência é a taxa com que elas são tocadas: $f_{ouvida} = f_{original} \times f_{s,reprodução} / f_{s,gravação}$. Tocar com o dobro da taxa dobra a frequência (sobe uma oitava, a "voz de esquilo") e corta a duração pela metade, como um disco rodando no dobro da velocidade.

PARA O QUIZ

No projeto isso significa: o emissor **precisa** tocar com a mesma f_s com que gerou o vetor (`sd.play(tom, 44100)`). Tocar o vetor gerado a 44 100 Hz com `sd.play(tom, 88200)` faria a tecla 7 sair como 1704 + 2418 Hz, que não é tecla nenhuma.

11 Revisão para o quiz

Fórmulas, pegadinhas e perguntas com resposta

Cola de fórmulas

Grandeza	Fórmula	Exemplo do projeto
Período	$T = 1/f$	852 Hz → 1,17 ms
Senoide	$s(t) = A \cdot \sin(2\pi f t + \varphi)$	A = 1, f = 852 Hz
Período de amostragem	$T_s = 1/f_s$	44 100 Hz → 22,7 µs
Número de amostras	$N = f_s \times$ duração	2 s → 88 200
Frequência de Nyquist	$f_s/2$ (exigência: $f_s > 2f_{max}$)	22 050 Hz
Resolução da FFT	$\Delta f = f_s/N = 1/T$	2 s → 0,5 Hz
Frequência do bin k	$f_k = k \cdot f_s/N$	bin 1704 → 852 Hz (2 s)
Níveis de quantização	2^n	16 bits → 65 536
Resolução do ADC	Δ = faixa / 2^n	100 mV, 14 bits → 6,1 µV
Erro máximo de quantização	$\Delta/2$	—
Taxa de bits	$f_s \times n_{bits} \times$ canais	CD: 1,41 Mbit/s
Altura do pico (Hamming)	$\lvert X\rvert \approx A \cdot N/2 \times 0{,}54$	N = 220 500 → ≈ 59 500
Decibéis	$20 \cdot \log_{10}(A_1/A_2)$ amplitude; $10 \cdot \log_{10}(P_1/P_2)$ potência	10× na amplitude = 20 dB
Alias	$\lvert f - k \cdot f_s\rvert$	11 Hz com f_s = 10 → 1 Hz
Reproduzir em outra taxa	$f' = f \times f_{s,rep}/f_{s,grav}$	4400 × 2 = 8800 Hz
Overhead (aula 5)	total / payload	1,1 → 10%

Pegadinhas

- A tabela do enunciado tem **679/825 Hz**; o certo é **697/852 Hz**. O **X** é o *****.
- Duas senoides de amplitude 1 somam até **2**: tocar sem dividir causa **clipping** e picos falsos.
- O espectro de um sinal real é **simétrico**: só metade (0 a $f_s/2$) tem informação.
- **Resolução depende da duração** ($\Delta f = 1/T$), não da taxa de amostragem.
- `min_dist` é em **bins**; `thres` é **relativo** (0 a 1).
- A **fase** não importa para identificar a tecla: usamos $\lvert X(f)\rvert$.
- Não pegue "os dois maiores picos": pegue **um de cada grupo**.
- `sd.play` e `sd.rec` **não bloqueiam**; `sd.wait()` bloqueia.
- `sd.rec` devolve uma matriz **(N, canais)**, não um vetor.
- Emissor a 44,1 kHz e receptor a 48 kHz funcionam juntos: frequência é grandeza física, em Hz.
- Número de bits arredonda **para cima**; 1 byte = 8 bits; 1 hora = 3600 s.

Perguntas e respostas

1. O que significa DTMF?

Dual-Tone Multi-Frequency: cada tecla é representada pela soma de dois tons, um do grupo baixo (linha) e um do grupo alto (coluna).

2. Quais as frequências da tecla 5? E da tecla 0?

5 = 770 Hz + 1336 Hz. 0 = 941 Hz + 1336 Hz.

3. Que tecla é 941 Hz + 1477 Hz?

(última linha, terceira coluna).

4. Quantas frequências e quantas teclas o DTMF tem?

8 frequências (4 baixas + 4 altas) e 16 teclas (4 × 4).

5. Por que usar dois tons e não um?

Robustez: voz e música podem conter um tom isolado e "discar" sem querer (talk-off); dois tons de grupos diferentes, simultâneos e estáveis são muito improváveis. E 8 frequências geram 16 combinações.

6. Por que as frequências DTMF não são múltiplas umas das outras?

Para que harmônicos gerados por distorção não caiam sobre outra frequência válida e criem uma tecla errada (ex.: 2 × 697 = 1394 Hz fica entre 1336 e 1477).

7. Por que 44 100 Hz?

É o padrão de áudio (CD, placas de som). Pelo teorema de Nyquist representa até 22 050 Hz, acima do limite da audição (~20 kHz).

8. Qual a menor taxa de amostragem teórica para DTMF?

Mais que 2 × 1633 = 3266 Hz. A telefonia digital usa 8000 Hz.

9. Quantas amostras têm 5 s de tom? Quanta memória em float64?

5 × 44 100 = 220 500 amostras; × 8 bytes ≈ 1,76 MB (em .wav de 16 bits, ≈ 441 kB).

10. O que a FFT recebe e o que devolve?

Recebe N amostras (reais) no tempo e devolve N números complexos, um por frequência $f_k = k \cdot f_s / N$. O módulo é a intensidade e o ângulo, a fase; para sinal real só a metade positiva interessa.

11. Qual a diferença entre DFT e FFT?

Nenhuma no resultado: a FFT é um algoritmo rápido para calcular a DFT (~$N \cdot \log N$ operações em vez de ~N^2).

12. Gravando 2 s, qual a resolução em frequência? E 10 ms?

$\Delta f = 1/T$: 0,5 Hz com 2 s; 100 Hz com 10 ms.

13. Com 10 ms de gravação dá para distinguir 852 de 941 Hz?

Não: estão a 89 Hz um do outro, menos que Δf = 100 Hz (e com Hamming o pico é ainda mais largo).

14. Dobrar a taxa de amostragem melhora a resolução?

Não. Dobra N, mas $\Delta f = f_s/N = 1/T$ continua igual. Só aumenta a frequência máxima.

15. Por que o gráfico do tempo mostra só alguns milissegundos?

Com segundos de sinal são centenas de milhares de pontos e milhares de ciclos; o gráfico vira uma mancha e não se vê a forma da onda.

16. Por que dividir a soma por 2 antes de tocar?

A soma vai de −2 a 2 e a placa aceita só −1 a 1. Acima disso a onda é cortada (clipping), o que cria harmônicos e intermodulação: picos falsos no receptor.

17. Para que serve a janela de Hamming? Qual o custo?

Reduz o vazamento espectral causado por cortar o sinal num trecho finito (lóbulos de −13 dB para −43 dB). O custo é um pico principal mais largo e 0,54× mais baixo.

18. O que são thres e min_dist no peakutils.indexes?

thres: altura mínima do pico como fração (0 a 1) da faixa máximo−mínimo. min_dist: distância mínima entre picos, em amostras (bins) do vetor.

19. Como garantir pelo menos 5 picos?

Diminuindo thres (aumentando a sensibilidade) até o detector achar 5.

20. Por que não usar simplesmente os dois maiores picos?

O segundo maior pode ser ruído ou um pico vizinho do primeiro. Como toda tecla tem exatamente um tom de cada grupo, escolhemos o mais forte perto de cada grupo.

21. Por que procurar picos só entre 500 e 2000 Hz?

Todo o DTMF está entre 697 e 1633 Hz. Abaixo há zumbido da rede (60 Hz e harmônicos), ventoinhas, voz grave; acima, harmônicos.

22. O que é o pico em 60 Hz no Fourier recebido?

Interferência da rede elétrica (60 Hz no Brasil), captada pelo microfone e pelos cabos.

23. Por que o pico aparece em 851,9 Hz e não em 852 Hz?

Os bins são discretos (resolução Δf), os relógios das placas de som diferem um pouco e há ruído. Por isso há tolerância (2%).

24. O que é aliasing e como evitar?

Uma frequência acima de $f_s/2$ aparece como outra, mais baixa e falsa. Evita-se amostrando a $f_s > 2f_{max}$ e filtrando antes do ADC (anti-aliasing).

25. Um áudio gravado a 40 kHz é tocado a 80 kHz. O que acontece?

Todas as frequências dobram (sobe uma oitava) e a duração cai pela metade. 4400 Hz vira 8800 Hz.

26. Que relação há entre a altura do pico e a amplitude do tom?

Proporcional: $|X| \approx A \cdot N/2 \times 0{,}54$ com Hamming. A = 1 e N = 220 500 → ~59 500.

27. O que sd.rec devolve e como tratar?

Uma matriz NumPy (numAmostras × canais). Com 1 canal: formato (N, 1); usa-se audio[:, 0] para ter um vetor.

28. Como o detector acerta com ruído 10× mais forte que o tom (SNR −10 dB)?

A FFT concentra a energia do tom num único bin, enquanto o ruído se espalha por dezenas de milhares de bins: ganho de processamento de $\approx 10 \cdot \log_{10}(N/2) \approx 46$ dB.

29. Quanto ocupa 1 h de áudio estéreo, 16 bits, 44,1 kHz?

44 100 × 16 × 2 × 3600 / 8 ≈ 635 MB.

30. Quantos bits para 10 µV de resolução numa faixa de 100 mV?

$2^n \geq 10\,000$ → n = 14 bits (resolução real 6,1 µV).

31. Quantos comparadores tem um ADC flash de n bits?

$2^n - 1$. Com 3 bits, 7 (o circuito da lista).

32. Quais são as três etapas da digitalização?

Amostragem (tempo discreto), quantização (amplitude discreta, com erro de até Δ/2) e codificação (nível → binário de n bits).

A Apêndice: código completo

Os três arquivos novos da pasta Projeto6_DTMF

suaBibSignal.py

```
# Biblioteca de apoio do Projeto 6 (DTMF).
#
# Reescrita da "suaBibSignal" cedida em aula, mantendo a mesma interface
# (classe signalMeu com generateSin, calcFFT e plotFFT), com três correções:
#   - a janela de Hamming vem de np.hamming (scipy.signal.hamming foi removida
#     do SciPy a partir da versão 1.13 e quebra o código original);
#   - o eixo de frequências usa np.fft.rfftfreq, que dá exatamente k*fs/N
#     (o linspace(0, fs/2, N//2) original fica levemente desalinhado);
#   - o vetor de tempo usa passo exato 1/fs (np.arange(n)/fs).

import numpy as np
import matplotlib.pyplot as plt
# ------------------------------------------------------------------- tabela DTMF
#
# Valores do padrão DTMF (ITU-T Q.23). A tabela do enunciado mostra 679 Hz e
# 825 Hz, que são 697 Hz e 852 Hz com os dígitos trocados: o próprio exemplo de
# saída do enunciado usa "1209 + 852" para a tecla 7.
# A tecla "X" do enunciado é o asterisco do teclado telefônico.

FREQS_BAIXAS = (697, 770, 852, 941)        # linhas do teclado
FREQS_ALTAS = (1209, 1336, 1477, 1633)     # colunas do teclado

TECLADO = (
    ("1", "2", "3", "A"),
    ("4", "5", "6", "B"),
    ("7", "8", "9", "C"),
    ("*", "0", "#", "D"),
)

# tecla -> (frequência baixa, frequência alta)
TABELA_DTMF = {
    TECLADO[i][j]: (FREQS_BAIXAS[i], FREQS_ALTAS[j])
    for i in range(4)
    for j in range(4)
}

# (frequência baixa, frequência alta) -> tecla
TECLA_POR_FREQS = {freqs: tecla for tecla, freqs in TABELA_DTMF.items()}
class signalMeu:

    def __init__(self):
        self.init = 0

    def generateSin(self, freq, amplitude, time, fs):
        """Senoide A*sin(2*pi*f*t) com `time` segundos amostrada a `fs` Hz."""
        n = int(round(time * fs))
        t = np.arange(n) / fs
        s = amplitude * np.sin(2 * np.pi * freq * t)
        return (t, s)

    def calcFFT(self, signal, fs):
        """Módulo da FFT (só frequências positivas) do sinal janelado.
```

```
        Retorna (xf, yf): xf são as frequências em Hz (passo fs/N) e yf o
        módulo |X(f)| correspondente. A janela de Hamming reduz o vazamento
        espectral causado por cortar o sinal num trecho finito.
        """
        signal = np.asarray(signal, dtype=float)
        N = len(signal)
        W = np.hamming(N)
        yf = np.fft.rfft(signal * W)
        xf = np.fft.rfftfreq(N, 1 / fs)
        return (xf, np.abs(yf))

    def plotFFT(self, signal, fs, titulo="Fourier", fmax=None):
        xf, yf = self.calcFFT(signal, fs)
        plt.figure()
        plt.plot(xf, yf, color="#2a78d6", linewidth=1)
        plt.title(titulo)
        plt.xlabel("Frequência (Hz)")
        plt.ylabel("|X(f)|")
        if fmax is not None:
            plt.xlim(0, fmax)
        plt.grid(True, color="#e1e0d9", linewidth=0.8)
        return xf, yf
```

encode.py

```
# Projeto 6 - DTMF - lado EMISSOR
#
# Pergunta uma tecla ao usuário, gera a soma das duas senoides DTMF dessa
# tecla, toca o som pela placa de som e plota o sinal no tempo e na frequência.
#
#   python encode.py                -> pergunta a tecla (repete até digitar q)
#   python encode.py 7              -> emite direto a tecla 7
#   python encode.py 7 --salvar     -> também grava tons/tecla_7.wav
#   python encode.py --duracao 8    -> muda a duração do tom (padrão 5 s)

import argparse
import os
import signal as sinais
import sys
import matplotlib.pyplot as plt
import numpy as np
import sounddevice as sd
from scipy.io import wavfile

from suaBibSignal import TABELA_DTMF, signalMeu

FS = 44100          # taxa de amostragem (amostras por segundo)
DURACAO = 5         # segundos de som
AMPLITUDE = 1       # amplitude de cada senoide
JANELA_PLOT = 0.02  # segundos mostrados no gráfico do tempo (20 ms)
FMAX_PLOT = 2000    # Hz mostrados no gráfico de Fourier
# funções caso queiram usar para sair...
def signal_handler(signal, frame):
    print('You pressed Ctrl+C!')
    sys.exit(0)


# converte intensidade em dB, caso queiram ...
def todB(s):
    sdB = 10 * np.log10(s)
    return sdB


def pedir_tecla():
```

```
    """Pergunta a tecla até o usuário digitar uma válida (ou q para sair)."""
    while True:
        tecla = input("Digite a tecla (0-9, A-D, *, #) ou q para sair: ").strip().upper()
        if tecla == "Q":
            return None
        if tecla == "X":            # o enunciado chama o * de X
            tecla = "*"
        if tecla in TABELA_DTMF:
            return tecla
        print("Tecla inválida.")


def gerar_tom(tecla, duracao=DURACAO, fs=FS):
    """Gera o tom DTMF da tecla. Retorna (t, s_baixa, s_alta, tom)."""
    signal = signalMeu()
    f_baixa, f_alta = TABELA_DTMF[tecla]
    t, s_baixa = signal.generateSin(f_baixa, AMPLITUDE, duracao, fs)
    _, s_alta = signal.generateSin(f_alta, AMPLITUDE, duracao, fs)
    tom = s_baixa + s_alta
    return t, s_baixa, s_alta, tom


def plotar(tecla, t, s_baixa, s_alta, tom, fs=FS):
    signal = signalMeu()
    f_baixa, f_alta = TABELA_DTMF[tecla]
    n = int(JANELA_PLOT * fs)
    t_ms = t[:n] * 1000
    # tempo: as duas senoides separadas e a soma, só nos primeiros 20 ms
    fig, ax = plt.subplots(3, 1, sharex=True, figsize=(9, 7))
    fig.suptitle(f"Tecla {tecla} - sinal no tempo (primeiros {JANELA_PLOT * 1000:.0f} ms)")
    ax[0].plot(t_ms, s_baixa[:n], color="#2a78d6", linewidth=1.5)
    ax[0].set_ylabel(f"{f_baixa} Hz")
    ax[1].plot(t_ms, s_alta[:n], color="#eb6834", linewidth=1.5)
    ax[1].set_ylabel(f"{f_alta} Hz")
    ax[2].plot(t_ms, tom[:n], color="#0b0b0b", linewidth=1.5)
    ax[2].set_ylabel("soma")
    ax[2].set_xlabel("tempo (ms)")
    for a in ax:
        a.grid(True, color="#e1e0d9", linewidth=0.8)

    # frequência: Fourier do sinal emitido
    xf, yf = signal.calcFFT(tom, fs)
    plt.figure(figsize=(9, 4))
    plt.plot(xf, yf, color="#2a78d6", linewidth=1.5)
    for f in (f_baixa, f_alta):
        k = np.argmin(np.abs(xf - f))
        plt.annotate(f"{f} Hz", (xf[k], yf[k]), textcoords="offset points",
                     xytext=(8, -4), color="#0b0b0b")
    plt.title(f"Tecla {tecla} - transformada de Fourier do sinal emitido")
    plt.xlabel("Frequência (Hz)")
    plt.ylabel("|X(f)|")
    plt.xlim(0, FMAX_PLOT)
    plt.grid(True, color="#e1e0d9", linewidth=0.8)
def salvar_wav(tecla, tom_saida, fs=FS):
    os.makedirs("tons", exist_ok=True)
    nome_tecla = {"*": "asterisco", "#": "sustenido"}.get(tecla, tecla)
    caminho = os.path.join("tons", f"tecla_{nome_tecla}.wav")
    wavfile.write(caminho, fs, (tom_saida * 32767).astype(np.int16))
    print(f"Sinal salvo em {caminho}")


def emitir(tecla, duracao, salvar):
    f_baixa, f_alta = TABELA_DTMF[tecla]
    print("Gerando Tons base")
    t, s_baixa, s_alta, tom = gerar_tom(tecla, duracao)

    # Cada senoide tem amplitude 1, então a soma vai de -2 a 2. A placa de som
```

```
    # só aceita valores entre -1 e 1: acima disso o sinal é cortado (clipping),
    # o que distorce a onda e cria frequências que não existiam. Dividir por 2
    # mantém o formato da onda e cabe na faixa.
    tom_saida = tom / 2

    print(f"Gerando Tom referente ao símbolo : {tecla}")
    print(f"Frequências : {f_baixa} + {f_alta} Hz")
    if salvar:
        salvar_wav(tecla, tom_saida)

    print(f"Executando as senoides (emitindo o som por {duracao} s)")
    sd.play(tom_saida, FS)      # não bloqueia: o som toca enquanto plotamos

    print("Plotando")
    plotar(tecla, t, s_baixa, s_alta, tom)
    plt.show()                  # bloqueia até as janelas serem fechadas

    sd.wait()                   # garante que o áudio terminou


def main():
    sinais.signal(sinais.SIGINT, signal_handler)

    parser = argparse.ArgumentParser(description="Emissor DTMF")
    parser.add_argument("tecla", nargs="?", help="tecla a emitir (sem ela, pergunta)")
    parser.add_argument("--duracao", type=float, default=DURACAO, help="segundos de som")
    parser.add_argument("--salvar", action="store_true", help="grava o tom em tons/*.wav")
    args = parser.parse_args()
    print("Inicializando encoder")
    if args.tecla is not None:
        tecla = args.tecla.strip().upper().replace("X", "*")
        if tecla not in TABELA_DTMF:
            sys.exit(f"Tecla inválida: {args.tecla}")
        emitir(tecla, args.duracao, args.salvar)
        return

    while True:
        print("Aguardando usuário")
        tecla = pedir_tecla()
        if tecla is None:
            break
        emitir(tecla, args.duracao, args.salvar)


if __name__ == "__main__":
    main()
```

decode.py

```
# Projeto 6 - DTMF - lado RECEPTOR
#
# Grava o som do microfone, calcula a transformada de Fourier, encontra os
# picos (pelo menos 5) e descobre qual tecla DTMF gerou o som.
#
#   python decode.py                          -> grava 2 s pelo microfone
#   python decode.py --duracao 3 --espera 5   -> grava 3 s depois de 5 s
#   python decode.py --arquivo tons/tecla_7.wav -> decodifica um .wav

import argparse
import time

import matplotlib.pyplot as plt
import numpy as np
```

```
import sounddevice as sd
from scipy.io import wavfile
from scipy.signal import find_peaks

from suaBibSignal import FREQS_ALTAS, FREQS_BAIXAS, TECLA_POR_FREQS, signalMeu

try:
    import peakutils
except ImportError:         # sem peakutils, usa o find_peaks do SciPy
    peakutils = None

FS = 44100          # taxa de amostragem
CANAIS = 1          # mono: uma lista só de amostras
DURACAO = 2         # segundos de gravação
ESPERA = 3          # segundos de contagem antes de gravar

F_MIN, F_MAX = 500, 2000    # faixa onde procuramos os picos (DTMF: 697-1633 Hz)
MIN_PICOS = 5               # o enunciado pede pelo menos 5 picos
MAX_PICOS = 10              # mostramos e usamos só os 10 mais fortes
DIST_PICOS_HZ = 5           # picos a menos de 5 Hz um do outro contam como um só
TOLERANCIA = 0.02           # pico vale como frequência DTMF se estiver a até 2% dela
FATOR_PISO = 5              # pico precisa ser 5x (14 dB) maior que o piso de ruído
# Sensibilidade do detector: começa exigente e afrouxa até achar MIN_PICOS picos.
# O último (0) aceita qualquer máximo local; só é preciso para sinais sem ruído.
LIMIARES = (0.5, 0.3, 0.2, 0.1, 0.05, 0.02, 0.01, 0.005, 0.002, 0.001, 0)


# função para transformar intensidade acústica em dB, caso queira usar
def todB(s):
    sdB = 10 * np.log10(s)
    return sdB


def gravar(duracao=DURACAO, espera=ESPERA, fs=FS):
    sd.default.samplerate = fs
    sd.default.channels = CANAIS
    numAmostras = int(duracao * fs)

    print(f"A captação começará em {espera} segundos. Dispare o emissor!")
    for falta in range(espera, 0, -1):
        print(f"  {falta}...")
        time.sleep(1)
    print("Gravação inicializada")
    audio = sd.rec(int(numAmostras), fs, channels=CANAIS)
    sd.wait()
    print("...     FIM")
    return audio


def ler_wav(caminho):
    fs, audio = wavfile.read(caminho)
    if np.issubdtype(audio.dtype, np.integer):
        audio = audio / np.iinfo(audio.dtype).max
    return audio, fs
def extrair_dados(audio):
    """A gravação pode vir como matriz (amostras x canais). Fica com um canal."""
    dados = np.asarray(audio, dtype=float)
    if dados.ndim > 1:
        dados = dados[:, 0]
    return dados - np.mean(dados)       # tira o nível DC do microfone


def trecho_com_som(dados, fs, quadro=0.02, limiar=0.3):
    """Índices (ini, fim) do trecho com som, descartando silêncio nas pontas.

    Divide a gravação em quadros de 20 ms e mantém do primeiro ao último
    quadro cuja energia (RMS) passa de 30% da maior energia.
```

```
    """
    n = int(quadro * fs)
    nq = len(dados) // n
    if nq < 2:
        return 0, len(dados)
    rms = np.sqrt(np.mean(dados[:nq * n].reshape(nq, n) ** 2, axis=1))
    ativos = np.nonzero(rms >= limiar * rms.max())[0]
    return ativos[0] * n, (ativos[-1] + 1) * n


def indices_de_picos(y, thres, min_dist):
    if peakutils is not None:
        return peakutils.indexes(y, thres=thres, min_dist=min_dist)
    altura = y.min() + thres * (y.max() - y.min())
    idx, _ = find_peaks(y, height=altura, distance=min_dist)
    return idx


def detectar_picos(xf, yf, fmin=F_MIN, fmax=F_MAX, min_picos=MIN_PICOS):
    """Picos de |X(f)| na faixa [fmin, fmax], do mais forte para o mais fraco.

    `thres` é a altura mínima do pico como fração do maior valor (0 a 1).
    Começamos exigentes e baixamos até aparecerem pelo menos `min_picos`.
    """
    faixa = (xf >= fmin) & (xf <= fmax)
    xb, yb = xf[faixa], yf[faixa]
    df = xf[1] - xf[0]
    min_dist = max(1, int(round(DIST_PICOS_HZ / df)))
    for thres in LIMIARES:
        idx = np.asarray(indices_de_picos(yb, thres, min_dist), dtype=int)
        if len(idx) >= min_picos:
            break
    idx = idx[np.argsort(yb[idx])[::-1]][:MAX_PICOS]
    piso = np.median(yb)
    return xb[idx], yb[idx], piso, thres


def mais_proxima(f, grupo):
    """Frequência DTMF do grupo mais próxima de f, ou None se longe demais."""
    alvo = min(grupo, key=lambda g: abs(g - f))
    return alvo if abs(alvo - f) <= TOLERANCIA * alvo else None
def identificar_tecla(freqs, mags, piso):
    """Escolhe o pico mais forte de cada grupo (baixo e alto) e acha a tecla.

    Toda tecla é exatamente uma frequência baixa (linha) + uma alta (coluna),
    então nunca escolhemos dois picos do mesmo grupo.
    """
    baixa = alta = None
    for f, m in zip(freqs, mags):           # já vem do mais forte ao mais fraco
        if m < FATOR_PISO * piso:
            continue                        # pico de ruído, fraco demais
        if baixa is None and mais_proxima(f, FREQS_BAIXAS):
            baixa = (mais_proxima(f, FREQS_BAIXAS), f, m)
        if alta is None and mais_proxima(f, FREQS_ALTAS):
            alta = (mais_proxima(f, FREQS_ALTAS), f, m)
    tecla = None
    if baixa and alta:
        tecla = TECLA_POR_FREQS[(baixa[0], alta[0])]
    return tecla, baixa, alta


def analisar(dados, fs):
    """Do sinal gravado até a tecla. Usado pelo main e pelos testes."""
    signal = signalMeu()
    ini, fim = trecho_com_som(dados, fs)
    xf, yf = signal.calcFFT(dados[ini:fim], fs)
    freqs, mags, piso, thres = detectar_picos(xf, yf)
```

```
    tecla, baixa, alta = identificar_tecla(freqs, mags, piso)
    return {
        "ini": ini, "fim": fim, "xf": xf, "yf": yf, "freqs": freqs,
        "mags": mags, "piso": piso, "thres": thres,
        "tecla": tecla, "baixa": baixa, "alta": alta,
    }


def imprimir(r):
    print(f"\nPicos encontrados (sensibilidade thres={r['thres']}), "
          "do mais forte para o mais fraco:")
    ref = r["mags"][0]
    for f, m in zip(r["freqs"], r["mags"]):
        db = round(20 * np.log10(max(m, 1e-300) / ref), 1) + 0.0  # 0 dB = mais forte
        print(f"   freq de pico = {f:8.1f} Hz   ({db:7.1f} dB)")
    if len(r["freqs"]) < MIN_PICOS:
        print(f"   (o espectro só tem {len(r['freqs'])} máximos locais na faixa: "
              "sinal sem nenhum ruído)")

    for nome, g in (("baixa", r["baixa"]), ("alta ", r["alta"])):
        if g:
            print(f"Frequência {nome}: {g[0]} Hz (pico em {g[1]:.1f} Hz)")
        else:
            print(f"Frequência {nome}: não encontrada")

    if r["tecla"]:
        print(f"\n==> TECLA IDENTIFICADA: {r['tecla']}")
    else:
        print("\n==> Nenhuma tecla DTMF reconhecida")


def plotar(dados, fs, r):
    t = np.linspace(0, len(dados) / fs, len(dados), endpoint=False)
    ini, fim = r["ini"], r["fim"]

    # tempo: gravação inteira (com o trecho usado destacado) e um zoom de 20 ms
    fig, ax = plt.subplots(2, 1, figsize=(9, 6))
    fig.suptitle("Sinal recebido no tempo")
    ax[0].plot(t, dados, color="#2a78d6", linewidth=0.6)
    ax[0].axvspan(t[ini], t[fim - 1], color="#2a78d6", alpha=0.10)
    ax[0].set_title("gravação inteira (faixa azul = trecho usado no Fourier)", fontsize=9)
    ax[0].set_xlabel("tempo (s)")
    meio = (ini + fim) // 2
    n = int(0.02 * fs)
    trecho = slice(meio, min(meio + n, len(dados)))
    ax[1].plot(t[trecho] * 1000, dados[trecho], color="#2a78d6", linewidth=1.5)
    ax[1].set_xlabel("tempo (ms) - zoom de 20 ms")
    for a in ax:
        a.grid(True, color="#e1e0d9", linewidth=0.8)
    fig.tight_layout()

    # frequência: Fourier com os picos encontrados e as 8 frequências DTMF
    xf, yf = r["xf"], r["yf"]
    fig, ax = plt.subplots(figsize=(9, 4.5))
    dtmf = FREQS_BAIXAS + FREQS_ALTAS
    for f in dtmf:
        ax.axvline(f, color="#c3c2b7", linewidth=0.8, zorder=0)
    topo = ax.secondary_xaxis("top")    # nomeia as linhas verticais DTMF
    topo.set_xticks(dtmf)
    topo.tick_params(labelsize=7, length=0)
    ax.plot(xf, yf, color="#2a78d6", linewidth=1.2, label="|X(f)|")
    ax.plot(r["freqs"], r["mags"], "o", color="#eb6834", markersize=6,
            label=f"{len(r['freqs'])} picos encontrados")
    for g in (r["baixa"], r["alta"]):
        if g:
            ax.annotate(f"{g[1]:.0f} Hz", (g[1], g[2]), textcoords="offset points",
                        xytext=(8, 0), color="#0b0b0b")
    titulo = f"tecla {r['tecla']}" if r["tecla"] else "nenhuma tecla"
    ax.set_title(f"Fourier do sinal recebido - {titulo}")
    ax.set_xlabel("Frequência (Hz)")
```

```
    ax.set_ylabel("|X(f)|")
    ax.set_xlim(0, 2000)
    ax.legend(loc="upper right")
    ax.grid(True, axis="y", color="#e1e0d9", linewidth=0.8)


def main():
    parser = argparse.ArgumentParser(description="Receptor DTMF")
    parser.add_argument("--arquivo", help="decodifica um .wav em vez de gravar")
    parser.add_argument("--duracao", type=float, default=DURACAO, help="segundos de gravação")
    parser.add_argument("--espera", type=int, default=ESPERA, help="segundos antes de gravar")
    args = parser.parse_args()

    if args.arquivo:
        audio, fs = ler_wav(args.arquivo)
        print(f"Lendo {args.arquivo} ({fs} Hz)")
    else:
        fs = FS
        audio = gravar(args.duracao, args.espera, fs)

    dados = extrair_dados(audio)
    r = analisar(dados, fs)
    imprimir(r)
    plotar(dados, fs, r)
    plt.show()


if __name__ == "__main__":
    main()
```